Dancing-Pipes

Project Documentation

Programmierprojekt, Sommersemester 2024

Atilla Özdemir
Kirill Dedov
Matsvei Tsiareshka
Toni Kostov

Table of contents

1. Motivation and goal setting

In the era of digitalization and technological innovation, controlling devices using gestures is becoming increasingly relevant. This is especially important for operating complex instruments such as an organ, which traditionally requires highly developed skills and physical interaction. Using computer vision to control an organ using gestures opens up new possibilities for a wider audience, including people without special training. This makes operating the organ accessible and intuitive, increasing convenience and efficiency. Since the project contains the most modern technologies, it was difficult for us, but we were inspired by the idea and were able to overcome everything

Objective of the project

The goal of our project is to develop a system for controlling an organ using gestures using computer vision technologies. We strive to create an interface that is simple and accessible to inexperienced users, allowing them to operate the organ easily and effectively. Our project aims to achieve the following key goals:

Intuitive Control: Developing a system that allows users to control the organ using simple and natural gestures, minimizing the need for special training.

Accessibility: Ensuring that organ operation is accessible to a wide range of people, including those without musical experience.

Innovation: Adopting advanced computer vision technologies to create an accurate and reliable gesture recognition system that provides smooth and natural control of the organ.

Our project aims to create an innovative solution that will make organ management accessible to everyone, regardless of level of training and experience. We believe that our system will open new horizons in the management of musical instruments, making this process more convenient, efficient and accessible. In the future, we hope that our project will be the beginning of the development of a system for playing the organ. So that people can use artificial intelligence to play famous songs and compose their own. However, this requires a large team and a group of music specialists.

2. Pose Estimation *(By Atilla and Matsvei)*

2.1. Evaluation of Tested Models

In this section, we compile and analyze the models for capturing and interpreting movements for the project, highlighting their advantages and disadvantages. The tests conducted here were carried out according to the specific purpose of the project and were evaluated in line with the limited resources of the developers. Therefore, it's important to note that these results may not apply universally to all scenarios.

This evaluation provides a comprehensive overview of the performance and suitability of each model in the context of motion detection, music track control, and data transmission, thereby guiding the selection of the most effective approaches for the system's development.

2.1.1. Kinect (Atilla and Matsvei)

The initial phase of the project explored the use of the Microsoft Kinect device for capturing and interpreting user movements. The Kinect offered a significant advantage:

- **3D Data Capture:** The Kinect's dual camera system and various sensors enabled the capture of 3D positional data of the user's body, providing detailed information for movement analysis.

However, the development process encountered several challenges:

- **Limited Compatibility:** The Kinect device's reliance on Windows operating systems posed a challenge for team members using different platforms.

- **Accessibility Issues:** The discontinuation of Kinect production made acquiring the device difficult, limiting the ability to support concurrent testing across multiple team members and significantly slowing down development.

Due to these accessibility limitations affecting both the development process and potential users, the project shifted focus towards more modern, versatile, and user-friendly solutions for capturing and interpreting movements.

2.1.2. Webcam-Based Pose Estimation with MoveNet (Atilla)

Following the exploration of the Kinect device, the project pivoted towards leveraging artificial intelligence-supported models for capturing and interpreting movements. This shift offered several advantages:

- **Accessibility:** Webcams are readily available on most personal computers, eliminating the need for additional sensors or specialized equipment. This significantly reduces barriers to both development and user adoption.

- **Cross-platform Compatibility:** Webcam functionality is inherent to most operating systems, ensuring broader compatibility compared to the Windows-specific Kinect.
- **Development Efficiency:** The widespread availability of webcams allows each team member to conduct independent testing and development cycles, accelerating the iterative process.

Given these advantages, the project explored MoveNet's Lightning model as the initial approach. This selection utilized the TensorFlow and OpenPose libraries, providing a well-established framework for pose estimation tasks. While initial tests with MoveNet demonstrated promising results in detecting the movements of a seated user, the model exhibited several limitations:

- **Limited Posture Recognition:** MoveNet struggled to accurately perceive the movements of a standing person with the same efficacy as those in a seated position. Extensive testing across various environments (lighting conditions, background variations) and user demographics (gender, height, skin tone) failed to yield significant improvements in this area.
- **Performance Issues with Speed:** The model's processing speed proved to be a bottleneck, particularly when dealing with fast movements or movements performed at low angles. These situations resulted in stuttering, slowdowns, and instances of missed detections.

Due to these limitations, the project redirected its focus towards finding alternative models that offered:

- **Lightweight Architecture:** A smaller, more efficient model would improve processing speed and address the performance issues encountered with MoveNet.
- **Posture Agnostic Detection:** The chosen model should be capable of accurately detecting and interpreting human movements regardless of the user's posture (standing or sitting).

By addressing these shortcomings, the project aims to establish a more robust and versatile solution for capturing and interpreting user movements.

2.1.3. Hand Recognition with Mediapipe+Tensorflow (Matsvei)

This model was not chosen due to the difficulty of combining with a pose recognition model. However, in the future they can be combined for more interactive use and a larger number of commands. ***See chapter 6.Hand gesture recognition***

2.1.4. Webcam-Based Pose Estimation with MediaPipe (Atilla)

Following the limitations encountered with MoveNet, the project explored MediaPipe as a potential alternative for capturing and interpreting user movements. MediaPipe shares some similarities with MoveNet in its model architecture but offers distinct advantages:

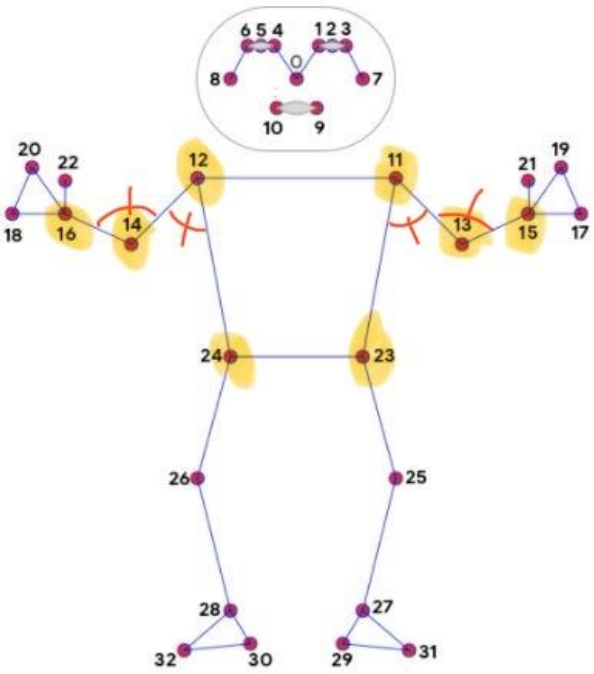
- **Simplified Installation:** MediaPipe boasts a streamlined installation process compared to MoveNet, reducing development setup time and streamlining project workflow.
- **Enhanced Performance:** Our evaluation revealed that MediaPipe outperformed MoveNet in terms of processing speed, frame capture rate, and overall detection accuracy. This translates to a smoother user experience with minimal latency and improved responsiveness to user movements.

However, during the evaluation process, one potential drawback of MediaPipe was identified:

- **Limited Customization:** While MediaPipe offers pre-built solutions for various tasks, its customizability for highly specialized applications might be less extensive compared to frameworks like TensorFlow.

Considering the project's time constraints and the positive results from the performance evaluation, MediaPipe was chosen as the preferred model for further development. However, the project acknowledges the need for potential future adjustments to address any limitations in customization or potential biases within the model.

2.2. Model Mechanism Overview (Atilla)



This section provides a comprehensive overview of how the developed model operates using MediaPipe and OpenCV to detect and interpret body movements for controlling a digital pipe organ. The system identifies 32 distinct points on the body, displaying both the detected points and the image to the user in a new pop-up window. In current version, 8 of these points are utilized for controlling the music system.

Detection and Display

Body Points Detection: The model

detects 32 distinct points on the user's body. These points are visualized on the screen, with 8 specific points used for the control mechanism.

- **Angle Calculation:** The coordinates of these 8 points are used to calculate angles, which are marked in red on the image. The relevant points for these calculations are highlighted in yellow.
- **Intermediate Codes Generation:** Based on the calculated angles, two intermediate codes are generated, one of these is for the tempo and the other is for the keyboard. These codes are displayed on the screen for user reference, allowing the user to monitor their current position.
- **Pose Detection:** If the user maintains a specific pose for more than 2 seconds, the intermediate codes of that pose are combined and transmitted to the server.

Control Mechanism

The control mechanism is divided into two primary functions: adjusting the tempo of the musical piece and changing the number of keyboards engaged. Additionally, a special pose controls the start and stop of the music track.

- **Tempo Control:**
  - The left side of the body is used for tempo control.
  - Angles centered on points 12 (shoulder) and 14 (elbow) are used to determine the control action.
  - Three different positions adjust the tempo:
    - **Decrease Tempo:** Lowers the current tempo by one unit.
    - **Increase Tempo:** Raises the current tempo by one unit.
    - **Default Tempo:** Resets the tempo to the default value.
- **Keyboard Control:**
  - The right side of the body manages the number of keyboards.
  - Angles centered on points 11 (shoulder) and 13 (elbow) are used to determine the control action.
  - Four different positions are defined for keyboard adjustments:
    - **Increase Keyboard:** Adds one to the number of keyboards.
    - **Decrease Keyboard:** Subtracts one from the number of keyboards.
    - **Max Keyboards:** Sets the number to the maximum value.
    - **Min Keyboards:** Sets the number to the minimum value.
- **Music Track Control:**
  - If both arms are lifted straight up in a certain pose, the server is notified that the player is ready to start the music track and ON/OFF appears on the screen.

- If this action is repeated after a music track has been started, the server is notified that the player wants to stop streaming and ON/OFF appears on the screen.

If the user presses the 'q' key, the camera detection process will halt.

Meanings of values

Elbow Angle	Shoulder Angle	Keyboard Value	Server Value	Tempo Value	Server Value
135°-180°	135°-180°	ON/OFF	0	ON/OFF	0
135°-180°	0°-45°	Ignored	Ignored	Ignored	Ignored
135°-180°	45°-90°	DELETE	6	MINUS	2
90°-135°	Insignificant	ADD	11	PLUS	3
45°-90°	Insignificant	MIN	16	Ignored	Ignored
0°-45°	Insignificant	MAX	21	DEFAULT	5

Recommended Setup and Conditions

To ensure optimal performance and avoid potential issues, the following preliminary steps are recommended:

- **Lighting:** Ensure that the light source shines from behind the camera towards the player.
- **Background:** Use a plain background that contrasts with the player's clothing and skin color.
- **Camera Distance:** Maintain a minimum distance between the player and the camera to ensure all body points are distinguishable.
- **Frame Rate:** For image detection to work well, a camera that supports 30 FPS images is required.

These setup recommendations help improve the accuracy of body point detection and enhance the overall functionality of the system.

3. Command Sending Logic *(By Matsvei)*

One of the critical aspects of this system is the efficient and accurate determination of commands based on user movements. This part provides an in-depth explanation of the logic behind sending commands to the server, combining note values, and maintaining a command history to ensure reliability and responsiveness.

Combining Note Value

The *combination_result* function is central to interpreting detected angles and translating them into actionable commands. The function takes two integer inputs, *manuale* and *tempo*, representing angles measured at the elbow and shoulder joints, respectively. These inputs are mapped to specific notes, and the combination of these notes forms a unique command.

```python
def combination_result(manuale, tempo):
    """
    Calculates a combined result based on two integer inputs: `manuale` and `tempo`.

    Arguments:
    manuale -- an integer representing a manual input (expected in the range 0-5).
    tempo -- an integer representing a tempo input (expected in the range 0-5).

    Returns:
    result -- an integer calculated as (manuale - 1) * 5 + tempo, or 0 if both inputs are 0.
    """
    # Convert inputs to int
    manuale = int(manuale)
    tempo = int(tempo)

    # Special case: if both inputs are zero, return 0
    if manuale == 0 and tempo == 0:
        return 0
    # Ensure inputs are within the valid range
    if 1 <= manuale <= 5 and 1 <= tempo <= 5:
        # Calculate the result based on the given formula
        return (manuale - 1) * 5 + tempo
```

Thus, using this function, two numbers are converted into one (according to the Meanings of values table) for further sending to the server. There are 26 different combinations in total. However, in this version of the project we only need 9 (see *Main Video Processing Loop*)

Sending to the server

The function *send_command_to_server* handles the communication with the server. It sends the command determined by the combination of *manuale* and *tempo* values.
The function sends an *HTTP POST* request to the server specified by the URL. The command is sent in the *JSON* format, making it easy for the server to parse and interpret the data.
The function includes a try-except block to handle any potential network errors. If an exception occurs, such as a connection timeout or server unavailability, it prints an error

message. This is important for debugging and ensuring the system can gracefully handle communication issues without crashing.

```
def send_command_to_server(url, command):
    """
    Sends a command to the server at the specified URL.

    Arguments:
    url -- the URL of the server.
    command -- the command to be sent to the server.
    """
    try:
        # Send a POST request to the server with the command in JSON format
        requests.post(url, json={'number': command})
    except requests.exceptions.RequestException as e:
        # Print error message if an exception occurs
        print(f"Error sending notes to server: {e}")
```

Main Video Processing Loop

To ensure that the commands sent to the server are reliable and reflect intentional user actions, the system maintains a history of recent commands using a *deque* (double-ended queue). This mechanism helps filter out accidental movements and ensures that only consistent and repeated actions trigger a command.

The *command_history* deque is initialized with a maximum length of 25. This means it will store the last 25 commands detected by the system. The choice of 25 is a balance between responsiveness and noise filtering, ensuring that the system reacts quickly to intentional movements while ignoring brief or accidental actions.

The *last_command* variable plays a key role in the logic of sending commands to the server. It is used to keep track of the last command sent and prevent duplicate commands from being sent without explicit modification.

The *command_allowed* variable is used to control the state of the system, determining when it can send commands to the server and when it cannot.

Since this version of the project has only 8 supported commands, the system first checks whether the command is supported. After this, the system adds the command number to the history

```
# Check if the command in allowed list
if command in {0, 2, 3, 5, 6, 11, 16, 21}:
    #add the allowed command to history
    command_history.append(command)
```

To indicate the beginning and end of the game, there was a designated start/end pose (arms up). It is designated by command 0. The system checks whether the deck is filled with number 0 and whether 0 is not the last command. The status of the variable *command_allowed* changes. If this position is used for the first time, then “ready” is printed and a thread is started that sends the value 0 to the server. If this pose is used a

second time, “end” is printed and a thread is started that sends the value 26 to the server. After this the history is cleaned

```python
# Check if the last 25 commands are all zero to toggle command state
if len(command_history) == 25 and all(cmd == 0 for cmd in command_history) and last_command != command:
    command_allowed = not command_allowed
    last_command = 0

    # Send commands based on the toggle state
    if command_allowed:
        print("ready")
        threading.Thread(target=send_command_to_server, args=(url, 0)).start()
    else:
        print("end")
        threading.Thread(target=send_command_to_server, args=(url, 26)).start()
    command_history.clear()
```

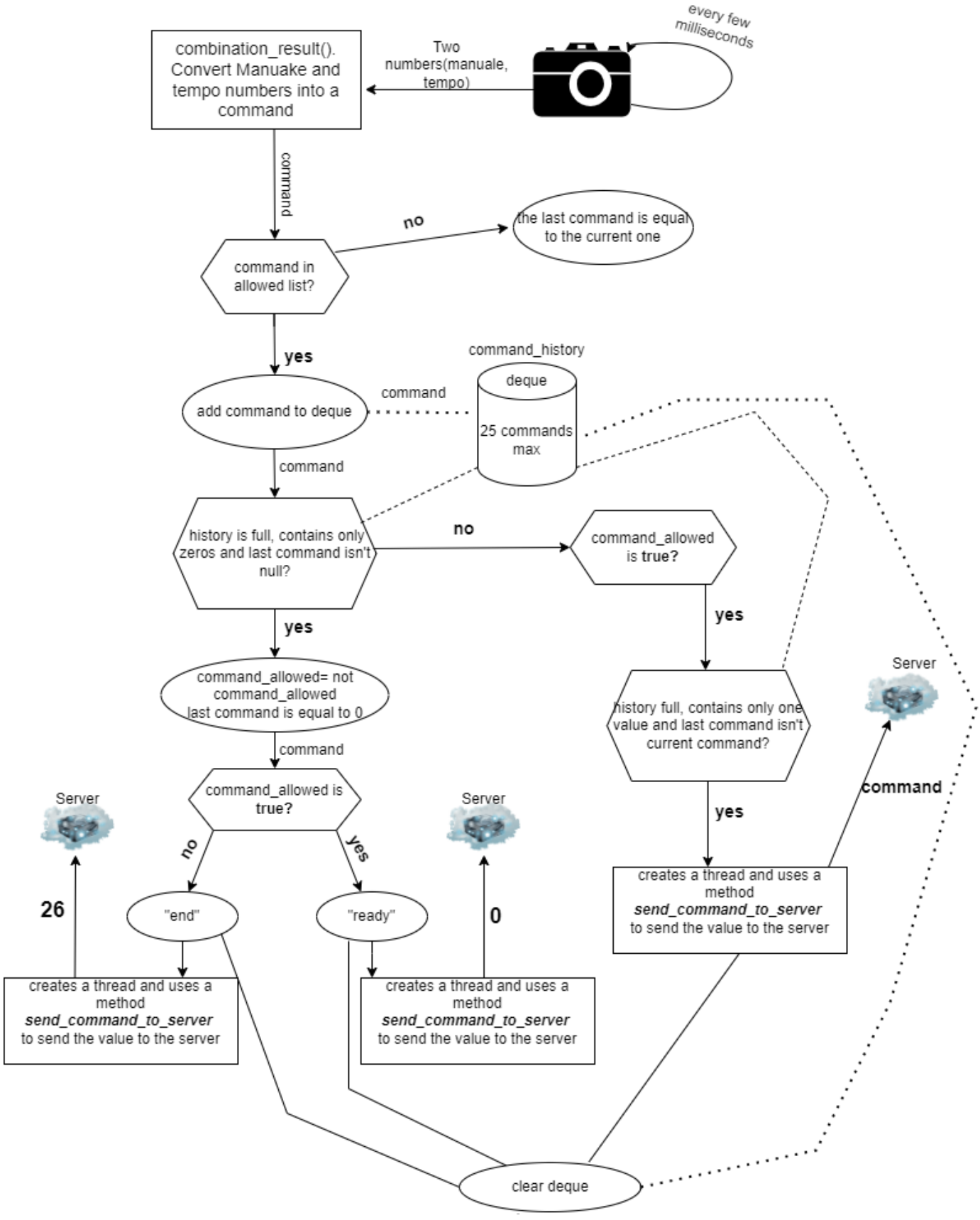
The system checks each time whether the starting pose has already been used. If so, then the history begins to fill with commands. When the history is filled with the same command, this command is sent to the server, the history is cleared

```python
# Send command if it filled out the history and allowed
if command_allowed:
    if len(command_history) == 25 and all(cmd == command_history[0] for cmd in command_history) and last_command != command:
        command_history.clear()
        threading.Thread(target=send_command_to_server, args=(url, command)).start()
        last_command = command
        print(command)
```

This section of the code is designed to ensure that commands sent to the server are intentional and consistent. The use of a deque to maintain command history, combined with multiple conditional checks, helps filter out accidental or transient movements. The toggling mechanism allows the system to switch between active and inactive states based on user poses, providing a robust and reliable method for controlling command sending.

Below you can see a visual representation of the algorithm

Command Sending Logic



4. Organ Server *(By Kirill)*

It is one of the components of the "Dancing Pipes." In our project, it connects the other components together and allows these components to be used on different computers.

4.1. Technologies and reasoning.

The server is built using the following main technologies:

- **Spring Boot:** Provides the framework for building and running the server, including support for web and web services.
    - *Reason:* Very popular framework that simplifies development with embedded servers, sensible defaults, and extensive ecosystem support. Additionally, gaining experience with Spring Boot is valuable for future projects and career growth.
- **Gradle:** Used as the build automation tool to manage dependencies and build processes.
    - *Reason:* Offers flexibility, incremental builds, and efficient dependency management. No special reasons for choosing Gradle, but I previously used Maven and now want to try something different.
- **JUnit:** Utilized for writing and running tests to ensure code quality and correctness.
    - *Reason:* Widely used, comprehensive features for testing, and strong IDE integration.
- **Lombok**: Helps reduce boilerplate code, improving code readability and maintainability.
    - *Reason:* Reduces boilerplate code, making the codebase more readable and maintainable.
- **Jackson:** Used primarily with ObjectMapper for JSON data binding, facilitating the conversion between Java objects and JSON.
    - *Reason:* High performance in parsing and generating JSON, and powerful and flexible ObjectMapper.
- **Spring Security Crypto:** Used to secure passwords in the login part of the frontend application, enhancing security in the application.
    - *Reason:* Provides robust mechanisms for securing passwords and seamless integration with Spring Security.
- **SseEmitter:** Utilized for server-sent events (SSE), enabling the server to push real-time updates to the client.
    - *Reason:* Ideal for real-time updates, reduces network overhead, and supports a push-based architecture for live notifications and data feeds. SSE is a simpler alternative to WebSockets for one-way communication, making it suitable for applications that require real-time data updates without the complexity of bidirectional communication. Additionally, since the goal was to build an HTTP server, SSE aligns well with this architecture.

4.2. Server Architecture

The main components of the server are three controllers: ConsumerController, ProducerController, and WebController; and four services: EmitterService, LoginService, NumberService, and OrganSettingsService.

4.2.1. Controllers

ConsumerController

The ConsumerController is a REST controller that handles incoming HTTP requests related to "Organ Sequencer" component. It is annotated with @RequiredArgsConstructor to automatically generate a constructor with required dependencies, which are EmitterService and NumberService. This controller provides functionality through two HTTP methods on the /consumer endpoint.

```java
@RequiredArgsConstructor  Dedov, Kirill *
@RestController
@RequestMapping("/consumer")
public class ConsumerController {

    private final EmitterService emitterService;
    private final NumberService numberService;

    @GetMapping  Dedov, Kirill *
    public ResponseEntity<SseEmitter> consumer() {
        try {
            SseEmitter emitter = emitterService.addConsumerEmitter();
            return ResponseEntity.ok(emitter);
        } catch (Exception e) {
            return ResponseEntity.internalServerError().body(null);
        }
    }

    @PostMapping  Dedov, Kirill *
    public ResponseEntity<ToConsumerDTO> adjustConfiguration(@RequestBody @Valid FromConsumerDTO config) {
        try {
            return numberService.getResponse(config);
        } catch (Exception e) {
            return ResponseEntity.internalServerError().body(null);
        }
    }
}
```

Endpoint: /consumer

HTTP Methods:

GET

- **Purpose:** This method is used to send commands to be executed in the "Organ Sequencer" component.
- **Method:** public ResponseEntity<SseEmitter> consumer()
- **Functionality:**

- When a GET request is made to this endpoint, it triggers the consumer().
- Consumer() attempts to create a new SseEmitter object via the EmitterService and returns it in the response.
- SseEmitter is used to send server-sent events to the client, which is useful for real-time updates.
- If the SseEmitter is created successfully, it returns an HTTP 200 OK response with the SseEmitter object. If an exception occurs, it returns a 500 Internal Server Error response with no body.

POST

- **Purpose:** This method is used to receive and adjust the configuration settings, which include information such as the number of keyboards, title name, and other related parameters.
- **Method:** public ResponseEntity<ToConsumerDTO> adjustConfiguration(@RequestBody @Valid FromConsumerDTO config)
- **Functionality:**
  - When a POST request is made to this endpoint with a valid FromConsumerDTO object in the request body, it triggers the adjustConfiguration method.
  - The adjustConfiguration method uses the NumberService to process the request and generate a response.
  - If successful, it returns a ToConsumerDTO object in the response. If an exception occurs, it returns a 500 Internal Server Error response with no body.

ProducerController

The ProducerController is a REST controller that handles incoming HTTP requests related to the "Camera" component. It is annotated with @RequiredArgsConstructor to automatically generate a constructor with required dependencies, which is NumberService. This controller provides functionality through a single HTTP method on the /producer endpoint.

```java
@RequiredArgsConstructor  Dedov, Kirill *
@RestController
@RequestMapping("/producer")
public class ProducerController {


    private final NumberService numberService;

    @PostMapping  Dedov, Kirill *
    public ResponseEntity<String> handleNumber(@RequestBody @Valid FromProducerDTO body) {
        try {
            numberService.sendNumber(body.number());
            return ResponseEntity.ok( body: "Processed number: " + body.number());
        } catch (Exception e) {
            return ResponseEntity.internalServerError().body("Error processing number: " + e.getMessage());
        }
    }
}
```

Endpoint: /producer

HTTP Methods:

POST

- **Purpose:** This method is used to receive commands from the "Camera" component and process them using the sendNumber method from NumberService.
- **Method:** public ResponseEntity<String> handleNumber(@RequestBody @Valid FromProducerDTO body)
- **Functionality:**
    - When a POST request is made to this endpoint with a valid FromProducerDTO object in the request body, it triggers the handleNumber method.
    - The handleNumber method uses the NumberService to process the number by calling the sendNumber method with the number from the request body.
    - If the number is processed successfully, it returns HTTP 200 OK with a message indicating the processed number. If an exception occurs, it returns a 500 Internal Server Error with an error message.

WebController

The WebController is a REST controller that handles incoming HTTP requests related to frontend. It is annotated with @RequiredArgsConstructor to automatically generate a constructor with required dependencies, which are LoginService and EmitterService. This controller provides functionality through two HTTP methods on the /web endpoint.

```java
@RequiredArgsConstructor  Dedov, Kirill *
@RestController
@RequestMapping("/web")
public class WebController {
    private final LoginService loginService;
    private final EmitterService emitterService;
    @GetMapping  Dedov, Kirill *
    public ResponseEntity<SseEmitter> streamWebNumbers() {
        try {
            SseEmitter emitter = emitterService.addWebClientEmitter();
            return ResponseEntity.ok(emitter);
        } catch (Exception e) {
            return ResponseEntity.internalServerError().body(null);
        }
    }


    @PostMapping("/login")  Dedov, Kirill
    public ResponseEntity<Boolean> login(@RequestBody UserCredentialsDTO credentials) {
        boolean loginSuccess = loginService.login(credentials.getUsername(), credentials.getPassword());
        if (loginSuccess) {
            return ResponseEntity.ok( body: true);
        } else {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(false);
        }
    }
}
```

Endpoint: /web

HTTP Methods:

GET

- **Purpose:** This method is used to send actual information to the frontend.
- **Method:** public ResponseEntity<SseEmitter> streamWebNumbers()
- **Functionality:**
    - When a GET request is made to the /web endpoint, it triggers the streamWebNumbers().
    - The streamWebNumbers() attempts to create a new SseEmitter object via the EmitterService and returns it in the response.
    - On Success: If the SseEmitter is created successfully, it returns an HTTP 200 OK response with the SseEmitter object.
    - On Failure: If an exception occurs, it returns a 500 Internal Server Error response with no body.

Endpoint: /web/login

HTTP Methods:

POST

- **Purpose:** This method is used to check user credentials.
- **Method:** public ResponseEntity<Boolean> login(@RequestBody UserCredentialsDTO credentials)
- **Functionality:**
    - When a POST request is made to the /web/login endpoint with a UserCredentialsDTO object in the request body, it triggers the login method.
    - The login method calls the loginService.login method, passing the username and password from the UserCredentialsDTO.
    - On Success: If the credentials are correct, it returns an HTTP 200 OK response with true.
    - On Failure: If the credentials are incorrect, it returns an HTTP 401 Unauthorized response with false.

4.2.2. Services

LoginService
The LoginService is a Spring service responsible for managing user authentication in the frontend. It achieves this by loading user credentials from a file and verifying login attempts using secure password hashing.
Key Fields:

- *users:* Stores usernames and their encoded passwords.
- *passwordEncoder:* Used to encode and verify passwords.
- *CREDENTIALS_FILE:* The name of the file that contains user credentials.

Methods

- *init()*
    - Purpose: Initializes the service by loading credentials from a file.
    - Annotation: @PostConstruct - This annotation ensures that the init() method is executed after the bean is constructed and dependencies are injected.
- *loadCredentialsFromFile()*

- Purpose: Loads user credentials from credentials.txt, encodes passwords, and stores them in a map.
- Throws: IOException - Propagates the exception to be handled by the caller.

- *login(String username, String password)*
  - Purpose: Verifies the provided username and password.
  - Returns: true if the credentials match; false otherwise.

EmitterService

The EmitterService is a Spring service responsible for managing Server-Sent Events (SSE) emitters for different types of clients. It handles the creation, management, and sending of events to these emitters.

Key Fields

- *consumerEmitter:* Manages SSE events for the "Organ Sequencer" client.
- *webClientEmitter:* Manages SSE events for frontend clients.
- *objectMapper:* Used to serialize messages into JSON format for sending via SSE.

Methods

- *addConsumerEmitter()*
  - Purpose: Initializes and returns a new SSE emitter for a “Organ Sequencer” client.
  - Returns: SseEmitter - a new emitter instance configured with maximum timeout and event handlers for completion, timeout, and error.

```
public SseEmitter addConsumerEmitter() {  1 usage   Dedov, Kirill
    consumerEmitter = new SseEmitter(Long.MAX_VALUE);
    consumerEmitter.onCompletion(() -> consumerEmitter = null);
    consumerEmitter.onTimeout(() -> consumerEmitter = null);
    consumerEmitter.onError(e -> consumerEmitter = null);
    return consumerEmitter;
}
```

- *addWebClientEmitter()*
  - Purpose: Initializes and returns a new SSE emitter for a frontend client.
  - Returns: SseEmitter - a new emitter instance configured with maximum timeout and event handlers for completion, timeout, and error.
- *sendToConsumer(ToConsumerDTO message)*

- Purpose: Sends a message to the “Organ Sequencer” client via SSE.
- Parameters:
- ToConsumerDTO message - the message to be sent to the “Organ Sequencer” client.

, it

- sendToWebClient(ToWebClientDTO message)
- Purpose: Sends a message to the web client via SSE.
- Parameters: ToWebClientDTO message - the message to be sent to the web client.
- Functionality: Sends the message directly. If an error occurs, it nullifies the web client emitter.

- *hasActiveConsumerEmitters()*
  - Purpose: Checks if there are any active consumer emitters.
  - Returns: boolean - true if there is an active consumer emitter, false otherwise.

NumberService

The NumberService is a Spring service that processes numerical commands from “Camera” component, updates organ settings, and communicates with clients via Server-Sent Events (SSE). It interacts with the EmitterService and OrganSettingsService to handle these tasks.

Key Fields

- *emitterService:* Manages the sending of messages to clients via SSE.
- *organSettingsService:* Manages the state and settings of the organ simulator.

Methods

- *sendNumber(int number)*
    - Purpose: Processes a numerical command to perform various actions related to organ settings and sends updates to clients.
    - Parameters: int number - the command number to process.
    - Functionality:
        - Determines the action based on the number using the Action enum.
        - Performs the corresponding action by calling methods in OrganSettingsService.
        - If the action results in a message, it updates and sends the message to the clients.

```
public void sendNumber(int number) {  1 usage  Dedov, Kirill
    Action action = Action.getAction(number);
    DTOWrapper message = switch (action) {
        case INCREMENT_KEYBOARDS -> organSettingsService.incrementKeyboards();
        case DECREMENT_KEYBOARDS -> organSettingsService.decrementKeyboards();
        case USE_ALL_KEYBOARDS -> organSettingsService.useAllKeyboards();
        case USE_ONE_KEYBOARD -> organSettingsService.useOneKeyboard();
        case INCREMENT_TEMPO -> organSettingsService.incrementTempo();
        case DECREMENT_TEMPO -> organSettingsService.decrementTempo();
        case DEFAULT_TEMPO -> organSettingsService.defaultTempo();
        case SEND_START_COMMAND -> organSettingsService.sendStartCommand();
        case SEND_STOP_COMMAND -> organSettingsService.sendStopCommand();
        case UNDEFINED -> null;
    };

    if (message != null) {
        updateAndSend(message);
    }
}
```

- *updateAndSend(DTOWrapper message)*
    - Purpose: Sends the message to the consumer and web client emitters if applicable.
    - Parameters: DTOWrapper message - the message to be sent.
    - Functionality:

- Sends the message to the consumer emitter if there are active consumer emitters.
- Checks conditions before sending the message to the web client emitter.

```
private void updateAndSend(DTOWrapper message) {  1 usage  Dedov, Kirill

    if (emitterService.hasActiveConsumerEmitters()) {
        emitterService.sendToConsumer(message.getToConsumerDTO());
    }
    if (sendCheck(message)) {
        emitterService.sendToWebClient(message.getToWebClientDTO());
    }

}
```

- *sendCheck(DTOWrapper message)*
  - Purpose: Checks whether the message should be sent to the web client.
  - Parameters: DTOWrapper message - the message to be checked.
  - Returns: boolean - true if the message should be sent; false otherwise.
  - Functionality:
    - Ensures the message is not sent if the consumer is connected and the command is either "start" or "stop".

- *getResponse(FromConsumerDTO config)*
  - Purpose: Updates organ settings based on the provided configuration and returns the updated settings to the consumer client.
  - Parameters: FromConsumerDTO config - the configuration settings to apply.
  - Returns: ResponseEntity<ToConsumerDTO> - the response containing the updated settings for the consumer client.
  - Functionality:
    - Updates the organ settings state based on the provided configuration.
    - Sends the updated state to the web client emitter.
    - Returns the updated configuration to the consumer client.

```
public ResponseEntity<ToConsumerDTO> getResponse(FromConsumerDTO config) {  1 usage   Dedov, Kirill
    organSettingsService.updateState(config);
    ToConsumerDTO toConsumerDTO = new ToConsumerDTO(organSettingsService.getKeyboardsInUse(),
            command: "configurationResponse", organSettingsService.getCurrentTempo());
    emitterService.sendToWebClient(organSettingsService.getMessage());
    return ResponseEntity.ok(toConsumerDTO);
}
```

OrganSettingsService

The OrganSettingsService is a Spring service responsible for managing the state and settings of an organ simulator. It includes functionality to increment and decrement keyboards and tempo, as well as to handle start and stop commands. The service communicates updates to clients via Server-Sent Events (SSE) using the EmitterService.

Key Fields

- *keyboardsInUse:* Number of keyboards currently in use.
- *maxAvailableKeyboards:* Maximum number of keyboards available.
- *startCommandReceived:* Boolean flag indicating if the start command was received.
- *minTempo:* Minimum allowable tempo.
- *maxTempo:* Maximum allowable tempo.
- *currentTempo:* Current tempo of the organ.
- *defaultTempo:* Default tempo setting.
- *barLength:* Length of the musical bar.
- *title:* Title of the current piece.
- *composerName:* Name of the composer.
- *message:* Current message to be sent to the web client.
- *command:* Current command name.
- *wasCommandExecuted:* Boolean flag indicating if the last command was executed.

Methods

- *setKeyboardsInUse(int keyboards)*
    - Purpose: Sets the number of keyboards in use if there are active consumer emitters.
    - Parameters: int keyboards - the number of keyboards to be set.

- *setMaxAvailableKeyboards(int keyboards)*

- Purpose: Sets the maximum number of available keyboards if there are active consumer emitters.
- Parameters: int keyboards - the maximum number of keyboards.

- *sendStartCommand()*
  - Purpose: Handles the start command and updates the state accordingly.
  - Returns: DTOWrapper - the updated state.

```
public DTOWrapper sendStartCommand() {  1 usage  Dedov, Kirill
    setCommand("start");
    if (isConsumerOnline() && !startCommandReceived) {
        setCurrentTempo(3);
        startCommandReceived = true;
        setCommandExecuted(true);
        return getCurrentValues();
    }

    setCommandExecuted(false);
    return getCurrentValues();
}
```

- *sendStopCommand()*
  - Purpose: Handles the stop command and updates the state accordingly.
  - Returns: DTOWrapper - the updated state.
- *incrementKeyboards()*
  - Purpose: Increments the number of keyboards in use if the start command was received and the maximum number of keyboards is not exceeded.
  - Returns: DTOWrapper - the updated state.

```
public DTOWrapper incrementKeyboards() {  1 usage  Dedov, Kirill
    setCommand("incrementKeyboards");
    if (startCommandReceived) {
        if (keyboardsInUse < maxAvailableKeyboards) {
            if (isConsumerOnline()) {
                keyboardsInUse++;
            }
            setCommandExecuted(true);
            return getCurrentValues();
        }
    }
    setCommandExecuted(false);
    return getCurrentValues();
}
```

- *decrementKeyboards()*
    - Purpose: Decrements the number of keyboards in use if the start command was received and more than one keyboard is in use.
    - Returns: DTOWrapper - the updated state.
- *useOneKeyboard()*
    - Purpose: Sets the number of keyboards in use to one if the start command was received and more than one keyboard is in use.
    - Returns: DTOWrapper - the updated state.
- *useAllKeyboards()*
    - Purpose: Sets the number of keyboards in use to the maximum available if the start command was received and not all keyboards are in use.
    - Returns: DTOWrapper - the updated state.
- *incrementTempo()*
    - Purpose: Increments the tempo if the start command was received and the maximum tempo is not exceeded.
    - Returns: DTOWrapper - the updated state.
- *decrementTempo()*
    - Purpose: Decrements the tempo if the start command was received and the minimum tempo is not exceeded.

- Returns: DTOWrapper - the updated state.

- *defaultTempo()*
  - Purpose: Sets the tempo to the default value if the start command was received and the current tempo is not the default.
  - Returns: DTOWrapper - the updated state.
  - getCurrentValues()
  - Purpose: Gets the current state of the organ settings and returns it wrapped in a DTOWrapper.

- *updateState(FromConsumerDTO config)*
  - Purpose: Updates the state of the organ settings based on the provided configuration.
  - Parameters: FromConsumerDTO config - the new configuration settings.
  - Returns: DTOWrapper - the current state.

```
public void updateState(FromConsumerDTO config) {  1 usage   Dedov, Kirill
    setMaxAvailableKeyboards(config.getKeyboardsMax());
    setKeyboardsInUse(config.getDefaultKeyboards());
    setTitle(config.getTitle());
    setBarLength(config.getBarLength());
    setComposerName(config.getComposerName());
    setMessage(getCurrentValues().getToWebClientDTO());
}
```

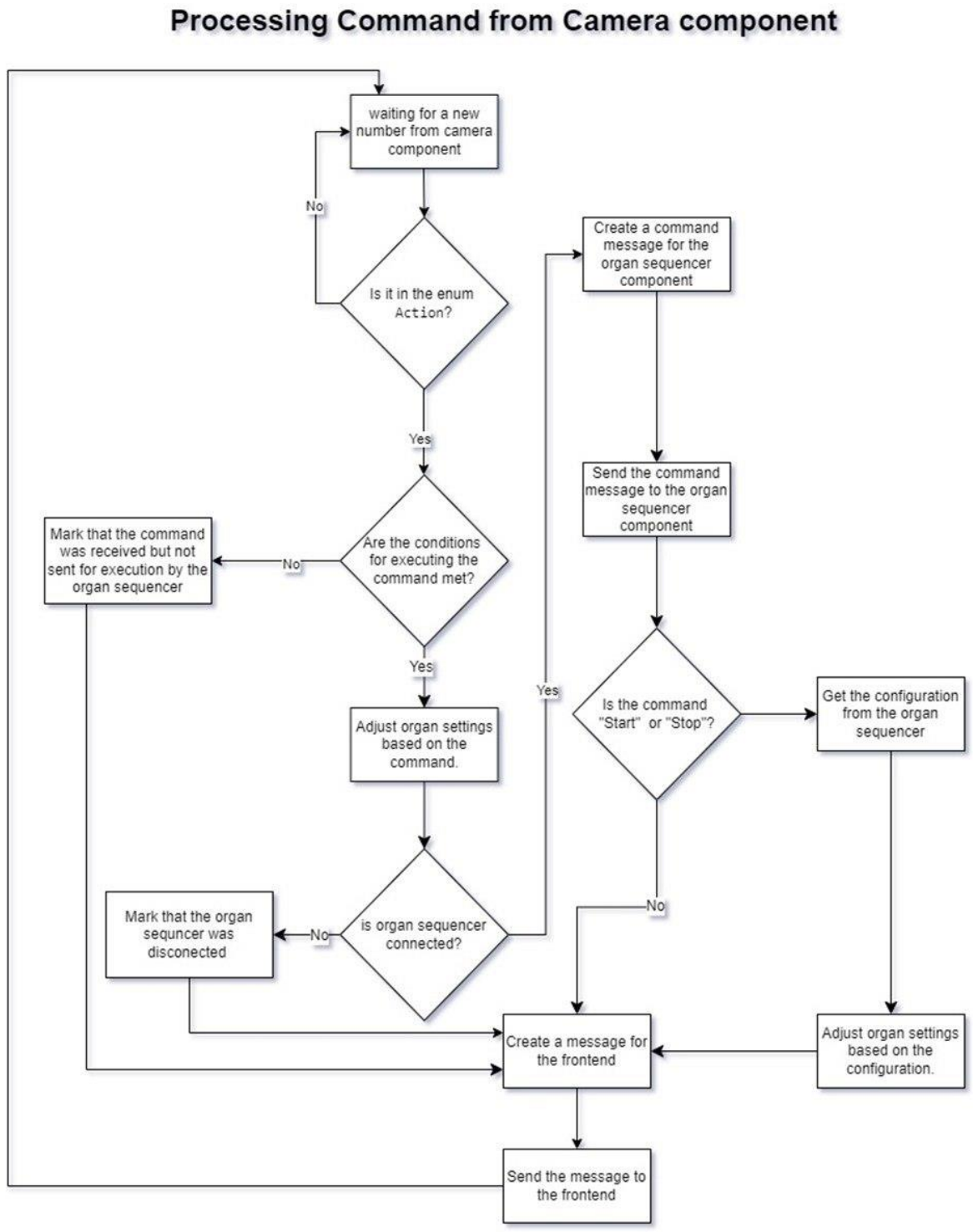
- *isConsumerOnline()*
  - Purpose: Checks if there are any active consumer emitters connected, indicating whether Organ Sequenser is connected and interacting with the server.
  - Returns: boolean - True if at least one consumer emitter is active, False otherwise.

- *getCurrentValues()*
  - Purpose: Retrieves the current state of the organ settings, wrapping them in a DTOWrapper. This includes both ToWebClientDTO and ToConsumerDTO, providing a snapshot of current configurations and states, useful for syncing states across components or updating client-side displays.
  - Returns: DTOWrapper - The wrapper containing the current settings and states, structured for both consumer and web client use.

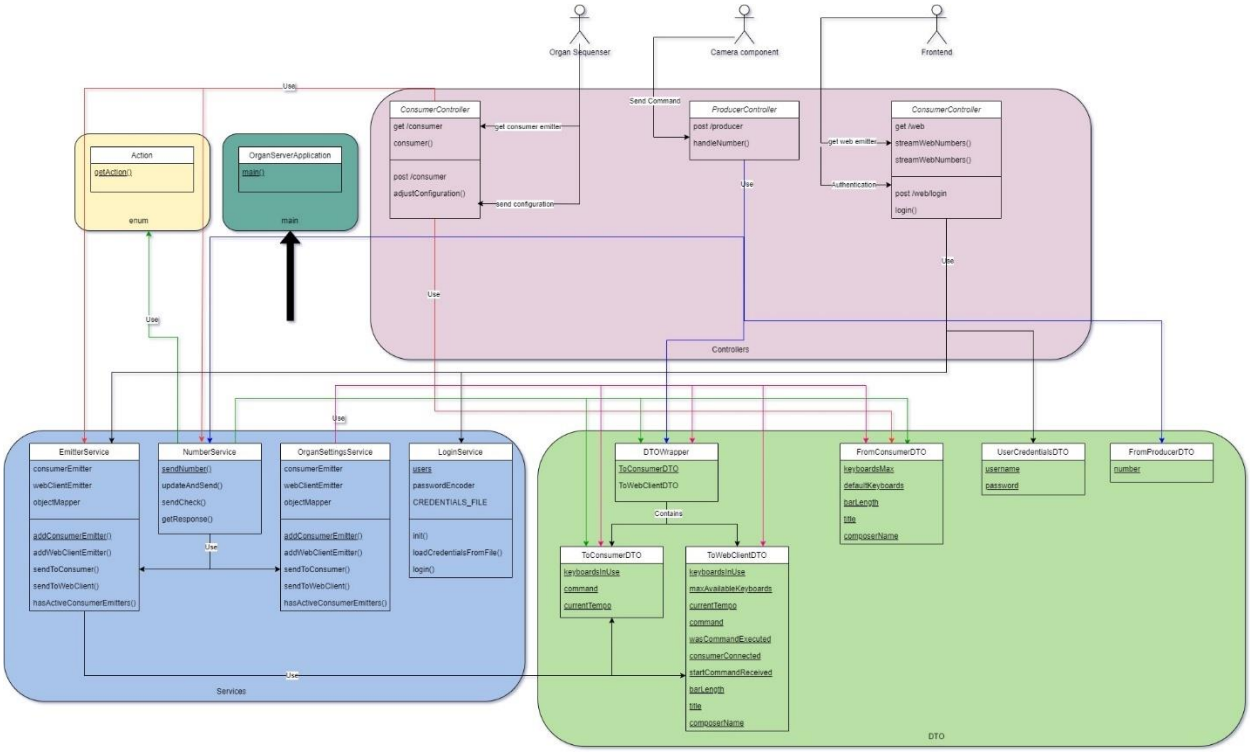
```
private DTOWrapper getCurrentValues() {  19 usages  Dedov, Kirill
    ToWebClientDTO toWebClientDTO;
    ToConsumerDTO toConsumerDTO = new ToConsumerDTO(getKeyboardsInUse(), getCommand(), getCurrentTempo());
    if (isConsumerOnline()) {
        toWebClientDTO = new ToWebClientDTO(getKeyboardsInUse(), getMaxAvailableKeyboards(), getCurrentTempo(),
                command, isCommandExecuted(), consumerConnected: true, isStartCommandReceived(), getBarLength(),
                getTitle(), getComposerName());
    } else {
        toWebClientDTO = new ToWebClientDTO(getKeyboardsInUse(), getMaxAvailableKeyboards(), getCurrentTempo(),
                command, wasCommandExecuted: false, consumerConnected: false, isStartCommandReceived(), getBarLength(),
                getTitle(), getComposerName());
    }

    return new DTOWrapper(toConsumerDTO, toWebClientDTO);
}
```

4.2.3. How the Server Handles Camera Commands

The task of the server is to receive signals from the "Camera" component, process them, decide whether they need to be sent to the "Organ Sequencer" component for execution, receive information from the "Organ Sequencer" about the piece being played, send information for display on the frontend, as well as store credentials and verify them for frontend login.



4.3. Class Diagram



5. Organ UI *(By Kirill)*

It is one of the components of the "Dancing Pipes." In our project, it facilitates a clearer visualization for users, showing the consequences of their commands issued via gestures in the camera component. This integration ensures that users can intuitively interact with the system, seeing the real-time impact of their actions on the interface.

5.1. Technologies and reasoning.

- **Angular:** It is popular for its powerful features like data-binding, which helps the app stay responsive and up-to-date without manual refreshes.
    - *Reason:* We use Angular as the main framework to build the frontend application. It's chosen for its strong ability to create dynamic single-page applications (SPAs). This framework is great for learning advanced web development skills too.
- **TypeScript:** It helps manage large projects by catching errors early in development, which saves time and improves the quality of the code. TypeScript adds rules about data types to JavaScript, making the code cleaner and reducing common errors.
    - *Reason:* We use TypeScript for writing the application because it comes with Angular.
- **Angular Material:** We incorporate Angular Material to make sure the app looks good and works well across different platforms.
    - *Reason:* Angular Material provides ready-to-use and good-looking UI components that help speed up development and improve the overall user experience.
- **RxJS:** This is used to handle data streams and updates in the app, important for keeping everything running smoothly and up-to-date.
    - *Reason:* It allows the app to handle multiple events and data changes efficiently, ensuring the interface reacts quickly to new information from the server.
- **Server-Sent Events (SSE):** We use SSE to get real-time updates from the backend, so the app always shows the latest data without needing to reload the page.
    - *Reason:* SSE is a simple way for the server to send new data to the app directly, which is great for applications where timely updates are crucial.

5.2. Frontend Architecture

In the frontend architecture of our project, we have structured the application using a variety of Angular components and services to ensure a cohesive and efficient user experience.
Components: LoginComponent, DashboardComponent, NumberDisplayComponent, OrganSettingsComponent, AppComponent.
Services: LoginDataService, SseService.

Guards: authGuard.
Pipes: CommandDisplayNamePipe, TempoPipe.
Enums: TempoLabels.
Intefaces: WebClientDTO.

5.2.1 Components

AppComponent
The AppComponent is the main part of the Angular application that manages the main page and how users move around in the app. It's set up to handle everything in one place, making it easier to manage.
Key Features:

- Updates Based on User Actions: It changes what it shows based on what the user is doing, like logging in or moving to different parts of the app.

Methods:

- *ngOnInit():* When the app starts, this method sets up to watch for changes in the user's login status and where they are in the app.
    - Purpose: Makes sure the app responds right when the user logs in or goes to other pages, updating things like welcome messages and links.
- *updateGreeting():* Changes the welcome message if the user is on the dashboard page, showing they're logged in.
    - Purpose: Gives a friendly message to logged-in users, making the app feel more personal.

HTML Structure:

- Includes a space to show a greeting to the user, which changes depending on if they're logged in.
- Shows or hides a "Log out" link based on whether the user is at the dashboard.
- Uses <router-outlet> to show content from other parts of the app depending on what the user chooses to do, which is central to how the app works.

```
<div class="header">
  <div class="user-name">{{ greeting }}</div>

  <h1>{{ title }}</h1>
  <a routerLink="/login" routerLinkActive="active">{{ linkText }}</a>
</div>

<router-outlet />
```

LoginComponent

The LoginComponent is an Angular component responsible for handling user authentication in the frontend of the application. It provides a form interface for users to enter login credentials and handles the submission of these credentials to verify user access.

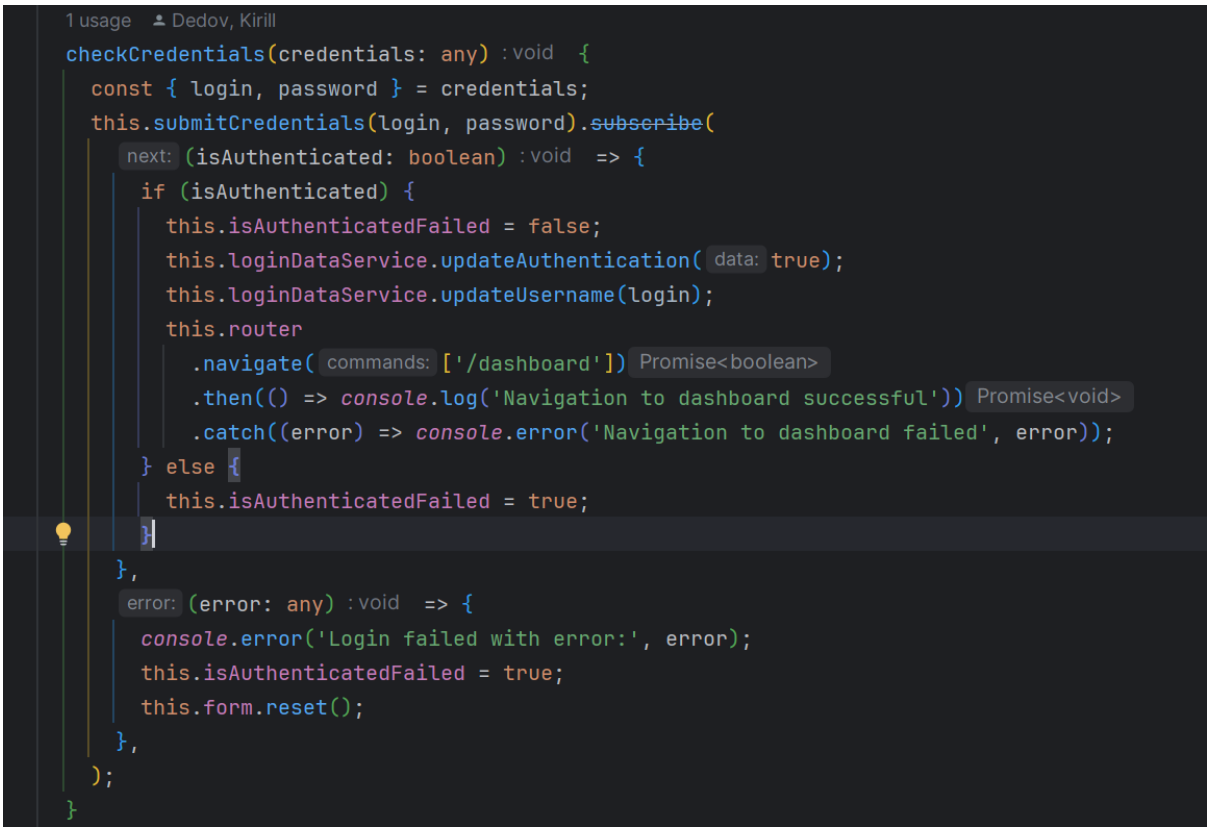
Key Properties:

- *form:* Holds the FormGroup instance that contains the login and password form controls.
- *isAuthenticatedFailed:* A boolean that flags whether authentication has failed, used to display error messages.
- *hide:* Controls the visibility of the password in the form, allowing users to toggle between showing and hiding their password.
- *apiUrl:* Stores the API URL from the environment settings, used to construct the request endpoint.

Methods:

- *ngOnInit():* Initializes the form with predefined validators.
    - Purpose: Sets up the form controls with required validations to ensure data integrity before submission.
- *toggleVisibility():* Toggles the hide property to switch the password input between text and password types.
    - Purpose: Enhances user experience by providing an option to view the password as they type.
- *onSubmit():* Called when the form is submitted; validates the form and proceeds to check credentials if the form is valid.
    - Purpose: Initiates the authentication process by verifying the user credentials against the backend.

- *checkCredentials(credentials: any):* Takes the username and password from the form, calls submitCredentials() to send them to the server, and handles the response.
    - Purpose: Authenticates the user by interacting with the backend API and updates the application state based on the authentication result.



- *submitCredentials(username: string, password: string):* Sends a POST request to the backend with the username and password.
    - Purpose: Directly communicates with the backend to verify user credentials.
    - Returns: Observable<boolean> indicating the success or failure of the login attempt.

DashboardComponent

The DashboardComponent serves as a central hub for the application's user interface, primarily responsible for aggregating and displaying major functional components related to the musical instrument control system. It integrates key components of the application, allowing for centralized and intuitive user interactions.

```
<div id="dashboardContent">
  <app-organ-settings></app-organ-settings>
  <app-number-display></app-number-display>
</div>
```

The template for the DashboardComponent is designed to serve as a container that embeds the OrganSettingsComponent and the NumberDisplayComponent. This layout ensures that users have immediate access to both the settings of the organ and the real-time feedback on number-related commands as they interact with the system.

The DashboardComponent acts as a critical element in the architecture, enabling efficient management of sub-components and ensuring that the user interface is not only functional but also aesthetically pleasing and user-friendly.

NumberDisplayComponent

The NumberDisplayComponent is an Angular component designed to display real-time command data related to the operation of the Organ Sequencer. It interfaces directly with server-sent events (SSE) to receive updates and present them to the user in an engaging and informative way.

Key Features:

- Dynamic Data Handling: The component subscribes to SSE updates via the SseService, ensuring that the data displayed is always current and reflects the latest server state.
- Reactive User Interface: Utilizes Angular's NgZone to ensure that state changes triggered by SSE updates are processed within the Angular's change detection context, ensuring smooth and performant UI updates.

HTML Structure:

- The component's template includes interactive elements such as buttons for toggling the view of command details and clearing the displayed data, enhancing user interaction.
- Commands are displayed in a list, with visual indicators for executed and non-executed commands, making it easy for users to understand the outcome of their actions.

Methods:

- *ngOnInit():* Sets up the subscription to the SSE data stream, initializing the component's state with live data from the server.
    - Purpose: Ensures that the component starts receiving and displaying data as soon as it is initialized.

```
no usages  Dedov, Kirill *
ngOnInit() : void  {
  this.sseService.getWebClientData().subscribe( observerOrNext: {
    next: (dto: WebClientDTO) : void  => {
      if (dto.keyboardsInUse <1) {
        this.zone.run( fn: () : void  => {
          this.webClientData.unshift(dto);
          this.updateDisplayedData();
        });
      }
    },
    error: (error) => console.error('Error received from SSE stream:', error),
  });
}
```

- *toggleView():* Allows users to toggle between viewing all commands or a limited set, enhancing usability for different user preferences.
    - Purpose: Provides flexibility in how data is displayed, making it easier for users to manage large volumes of command information.
- *clearData()*: Clears the current command data from the display, offering a way to reset the view or manage data visibility.
    - Purpose: Helps maintain a clean interface, particularly useful in scenarios where data needs to be refreshed or if the display becomes too cluttered.
- *updateDisplayedData():* Manages which data is shown based on user interactions, such as toggling between full and limited views.
    - Purpose: Dynamically adjusts the amount of data displayed, ensuring that the interface remains manageable and relevant to the user's current needs.

```
2 usages  Dedov, Kirill
private updateDisplayedData() : void  {
  this.displayedData = this.showAll ? [...this.webClientData] : this.webClientData.slice(0, this.maxDisplay);
}
}
```

OrganSettingsComponent
The OrganSettingsComponent is an Angular component dedicated to displaying and managing settings related to the organ's operational parameters. It provides a user interface for real-time interaction with the organ, such as adjusting keyboards and setting tempo, ensuring users have direct control over these aspects.
Key Features:

- Real-time Data Interaction: Subscribes to data updates from the SseService, ensuring that the component displays current information and reacts to changes as they occur.

HTML Structure:

- The component template includes sections for displaying composition details such as title, composer, and the number of bars. It also features visual indicators for the status of each keyboard and command received.
- Provides visual feedback on connectivity and command reception, with status bars that change colour based on the connection status and whether a start command has been received, thereby informing the user of the system's readiness to receive further commands.

Methods:

- *ngOnInit():* Initiates a subscription to the SseService to receive updates on the organ settings. It processes incoming data within Angular's NgZone to ensure proper UI rendering and state management.
    - Purpose: Keeps the component's state synchronized with the backend, providing users with up-to-date information and control capabilities.

```
ngOnInit(): void {
  this.subscription = this.sseService.getWebClientData().subscribe( observerOrNext: {
    next: (data: WebClientDTO) : void  => {
      this.ngZone.run( fn: () : void  => {
        this.webClientData = data;
        this.consumerIsConnected = data.consumerConnected;
        this.selectedTempoLabel = data.currentTempo;
        this.updateKeyboards();
        this.startCommandReceived = data.startCommandReceived;
        this.barLength = data.barLength;
        this.title = data.title;
        this.composerName = data.composerName;
      });
    },
    error: (error) => console.error('Error receiving SSE loginData:', error),
  });
}
```

- *updateKeyboards():* Dynamically adjusts the visual representation of the keyboards based on the current data, such as which keyboards are active, inactive, or disabled.
    - Purpose: Allows users to visually perceive the current configuration of the organ in real time, enhancing the interactive experience.

```
2 usages  Dedov, Kirill
updateKeyboards(): void {
  this.keyboards = Array( arrayLength: 5).fill( value: 'disabled');

  if (this.webClientData) {
    let max: number = this.webClientData.maxAvailableKeyboards < 0 ? 0 : this.webClientData.maxAvailableKeyboards;
    let inUse: number = this.webClientData.keyboardsInUse < 0 ? 0 : this.webClientData.keyboardsInUse;
    for (let i :number  = 0; i < inUse; i++) {
      this.keyboards[i] = 'active';
    }

    for (let i :number  = inUse; i < max; i++) {
      this.keyboards[i] = 'inactive';
    }
  }
}
```

- *getKeyboardImage(index: number):* Returns the appropriate image path based on the keyboard's status, aiding in creating a visually intuitive interface.
    - Purpose: Enhances the user interface by providing visual tipps that reflects the current state of each keyboard.

```
4 usages  Dedov, Kirill
getKeyboardImage(index: number): string {
  const status = this.keyboards[index];
  switch (status) {
    case 'active':
      return 'active keyboard.png';
    case 'inactive':
      return 'inactive keyboard.png';
    default:
      return 'disabled keyboard.png';
  }

}
```

- *getKeyboardName(index: number):* Maps each keyboard index to its corresponding name, ensuring accurate labeling and enhancing usability.
    - Purpose: Facilitates user understanding of the organ's configuration, making the interface more accessible and easier to navigate.

5.2.1 Services

SseService

The SseService is an Angular service designed to handle real-time data connections using Server-Sent Events (SSE). It's crucial for updating the application with live data from the server without needing to refresh the page. This service plays a key role in ensuring that the user interface displays the most current system information.

Key Features:

- Automatic Updates: The service uses SSE to listen for updates from the server and immediately updates the application's state with any new data.
- Central Data Management: Manages a central stream of data updates using a BehaviorSubject, which allows components throughout the application to access the latest data easily.

Methods:

- initEventSource(url: string): Initializes the SSE connection to the server.
    - Purpose: Opens a continuous channel between the server and the client for sending real-time updates. It also handles data parsing and updates the application state using Angular's NgZone to ensure UI updates are processed in the Angular context

Creation of Dummy State on Error: In the onerror event handler, it handles cases where no commands have been sent or when there's a problem with the connection. It sets up a basic fallback or "dummy" state with simple default values like command: '', currentTempo: 0, and title: 'stopped'. This smart move makes sure that the application stays stable and behaves as expected, even if no commands come through or if there are unexpected issues with data coming from the server.

```
1 usage   Dedov, Kirill *
private initEventSource(url: string): void {
  this.eventSource = new EventSource(url);
  this.eventSource.onmessage = (event : MessageEvent ) : void  => {
    this.zone.run( fn: () : void  => {
      try {
        const data : WebClientDTO  = JSON.parse(event.data) as WebClientDTO;
        this.dataSubject.next(data);
        console.log(data.composerName)
      } catch (error) {
        console.error('Error parsing loginData:', error);
        this.dataSubject.next(this.dataSubject.value);
      }
    });
  };

  this.eventSource.onerror = (error : Event ) : void  => {
    this.zone.run( fn: () : void  => {
      console.error('SSE error:', error);
      this.dataSubject.next( value: {
        command: '',
        currentTempo: 0,
        wasCommandExecuted: false,
        keyboardsInUse: 0,
        maxAvailableKeyboards: 0,
        consumerConnected: false,
        startCommandReceived: false,
        barLength: 0,
        title: 'stopped',
        composerName: 'stopped'
      });
    });
  };
}
```

- *getWebClientData(): Observable<WebClientDTO>:* Provides an observable that components can subscribe to receive updates.
    - Purpose: Allows any part of the application to listen for real-time data updates, ensuring the UI is always synchronized with the server state.

- *ngOnDestroy():* Cleans up the SSE connection when the service is destroyed.
    - Purpose: Prevents memory leaks and other performance issues by ensuring that the connection is properly closed when the service instance is no longer needed.

LoginDataService
The LoginDataService is a service, designed to manage user authentication.

Key Features:
- Reactive Data Streams: Utilizes BehaviorSubject from RxJS to handle changes in the user's authentication status and username. This reactive approach allows various parts of the application to immediately respond to any changes in user status.
- Centralized User State Management: Centralizes the management of user authentication and identity, simplifying the process of accessing and reacting to user data throughout the application.

Methods:
- *updateAuthentication(data: boolean):* Updates whether the user is authenticated.
    - Purpose: This method is invoked to reflect changes in the user's authentication status (logged in or logged out). It ensures that all parts of the application that rely on the user's authentication status receive updates in real-time.
- *updateUsername(newUsername: string):* Updates the stored username.
    - Purpose: This is used to set or update the username whenever a user logs in.

5.2.3. Guards

authGuard.

The authGuard. This guard ensures that only authenticated users can access dashboard.

```
export const authGuard: CanActivateFn = () => {
  const dataService : LoginDataService  = inject(LoginDataService);
  const router : Router  = inject(Router);

  return dataService.loginData.pipe(
    map( project: (isAuthenticated : boolean ) : UrlTree | boolean  => {
      if (!isAuthenticated) {
        return router.createUrlTree( commands: ['/login']);
      }
      return true;
    }),
  );
};
```

Integration with Routes:

- Dashboard Route with Guard: The authGuard is directly applied to the dashboard route in the application's routing configuration:

This setup means that when a user tries to navigate to the /dashboard route, the authGuard is invoked before the route is activated.

```
export const routes: Routes = [
  {
    path: 'login',
    loadComponent: () => import('./login/login.component').then((m :{LoginComponent: LoginComponent} ) => m.LoginComponent),
  },
  {
    path: 'dashboard',
    loadComponent: () => import('./dashboard/dashboard.component').then((m :{DashboardComponent: DashboardComponent} ) => m.DashboardComponent),
    canActivate: [authGuard],
  },
  {
    path: '',
    redirectTo: '/login',
    pathMatch: 'full',
  },
];
```

Functionality of authGuard in Routing:

- Authentication Check: When the authGuard is triggered by a route access attempt, it consults the LoginDataService to determine if the user is currently authenticated.
- Conditional Routing:
  - If the user is authenticated (isAuthenticated is true), the guard allows the routing to proceed, granting access to the dashboard.
  - If the user is not authenticated, the guard redirects the user to the /login route using a URL tree created by the Router. This redirection prevents access to the dashboard and guides the user to a place where they can log in.

5.2.4 Pipes

TempoPipe.

This pipe designed to transform tempo enum values into user-friendly string descriptions.

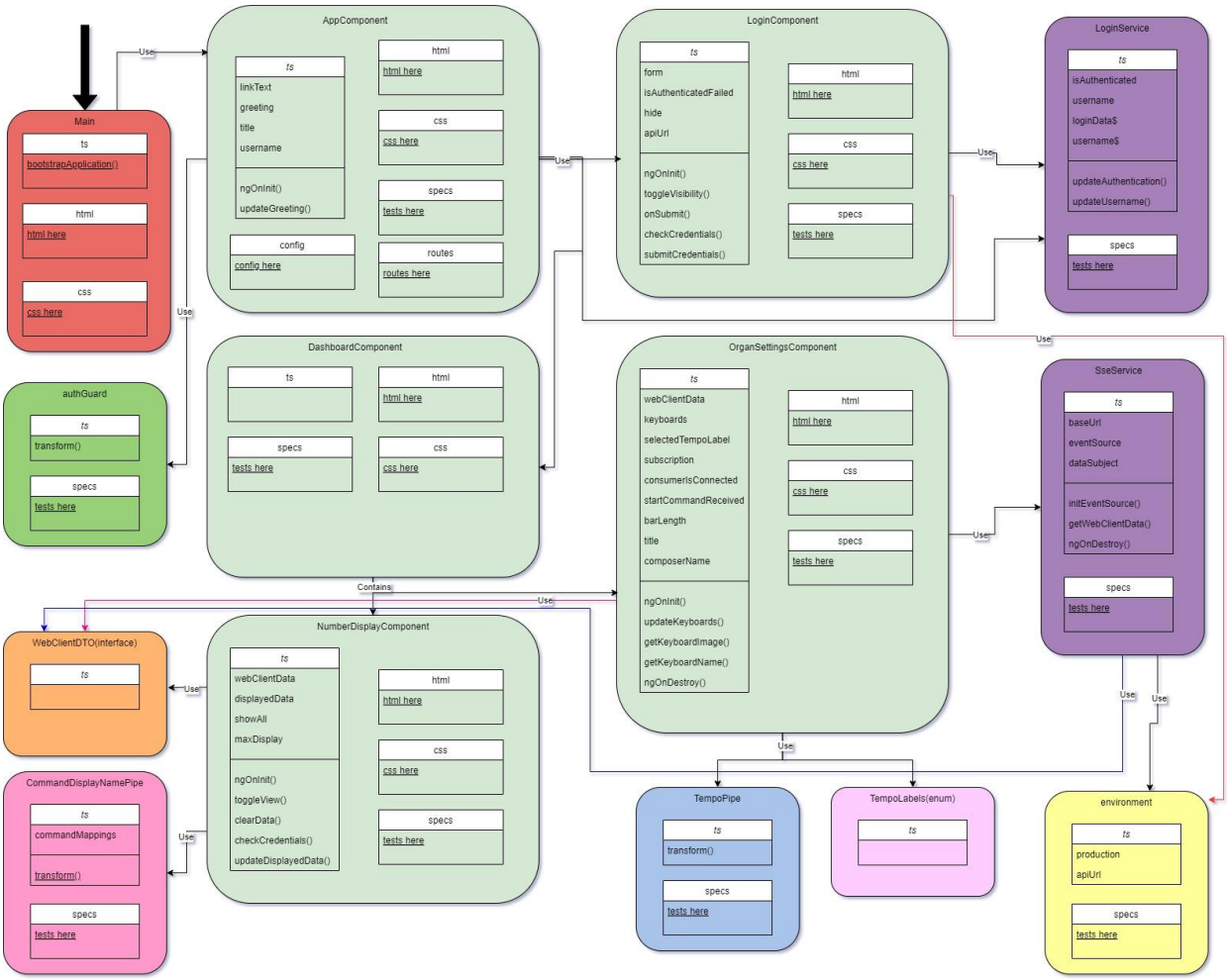
```
transform(tempo: TempoLabels) : string  {
  switch (tempo) {
    case TempoLabels.STOPPED:
      return 'Stopped';
    case TempoLabels.VERY_SLOW:
      return 'Very Slow';
    case TempoLabels.SLOW:
      return 'Slow';
    case TempoLabels.NORMAL:
      return 'Normal';
    case TempoLabels.FAST:
      return 'Fast';
    case TempoLabels.VERY_FAST:
      return 'Very fast';
  }
}
```

CommandDisplayNamePipe.

This pipe is designed to transform command keys into user-friendly string descriptions.

```
private commandMappings: { [key: string]: string } = {
  start: 'Start',
  stop: 'Stop',
  incrementKeyboards: 'Add One Manual',
  decrementKeyboards: 'Remove One Manual',
  minKeyboards: 'Use One Keyboard',
  maxKeyboards: 'Use All Keyboards',
  incrementTempo: 'Increase Tempo',
  decrementTempo: 'Decrease Tempo',
  defaultTempo: 'Reset to Default Tempo'
};

1 usage  Dedov, Kirill
transform(command: string): string {
  return this.commandMappings[command] || command;
}
```

5.3. Class Diagram



6. MIDI Sequencer with Client (by Toni)

6.1. General overview

The MIDI component of the project includes the management of music patterns (in MIDI file format) for each keyboard of the organ, alongside the creation of a MIDI sequencer and Client for connecting with the server to allow playing on the organ. The sequencer plays music while accepting control inputs — such as increasing or decreasing dynamics and adjusting tempo — via a camera. The MIDI component functions as a standalone application that can either connect as a client to a server, interfacing with the camera and front-end, or operate offline, allowing developers to test sequencer features using keyboard input. Additionally, it supports connections to various devices and virtual MIDI ports, though third-party software may be required for these connections (see Software dependencies section).

6.2. Development and Challenges

Initial Planning and Decisions

From the project's inception, it was clear that controlling the organ's music via MIDI signals was a fundamental requirement. Following a detailed discussion with Prof. Ritschel, we outlined the operations of the sequencer and identified key goals. Subsequently, we researched suitable programming languages and libraries, narrowing our choice to C++ (e.g., using the rtmidi library), Rust, and Java (MIDI package).

Language and Library Selection

Initially, C++ was considered due to its robust libraries, but many operated at a very low level, necessitating extensive manual handling of MIDI signals. Rust, while offering several MIDI libraries, was also ruled out due to our limited familiarity and personal preferences. Ultimately, we chose Java for its comprehensive MIDI package, our existing experience with the language, and its robust multithreading capabilities, crucial for real-time control of the sequencer.

Implementation and Challenges

Understanding the MIDI interface, message structure, and channel management required considerable effort. After grasping these concepts, we focused on developing the sequencer's logic, which, while demanding, was achievable.
One significant challenge was ensuring the sequencer sounded organic, particularly when adding patterns. I haven't been able to fully resolve this issue, which sometimes

results in abrupt sounds when adding patterns. Similarly, tempo changes can also sound unnatural, but this can be corrected by adjusting the tempo factor when configuring a composition.

6.3. Software dependencies

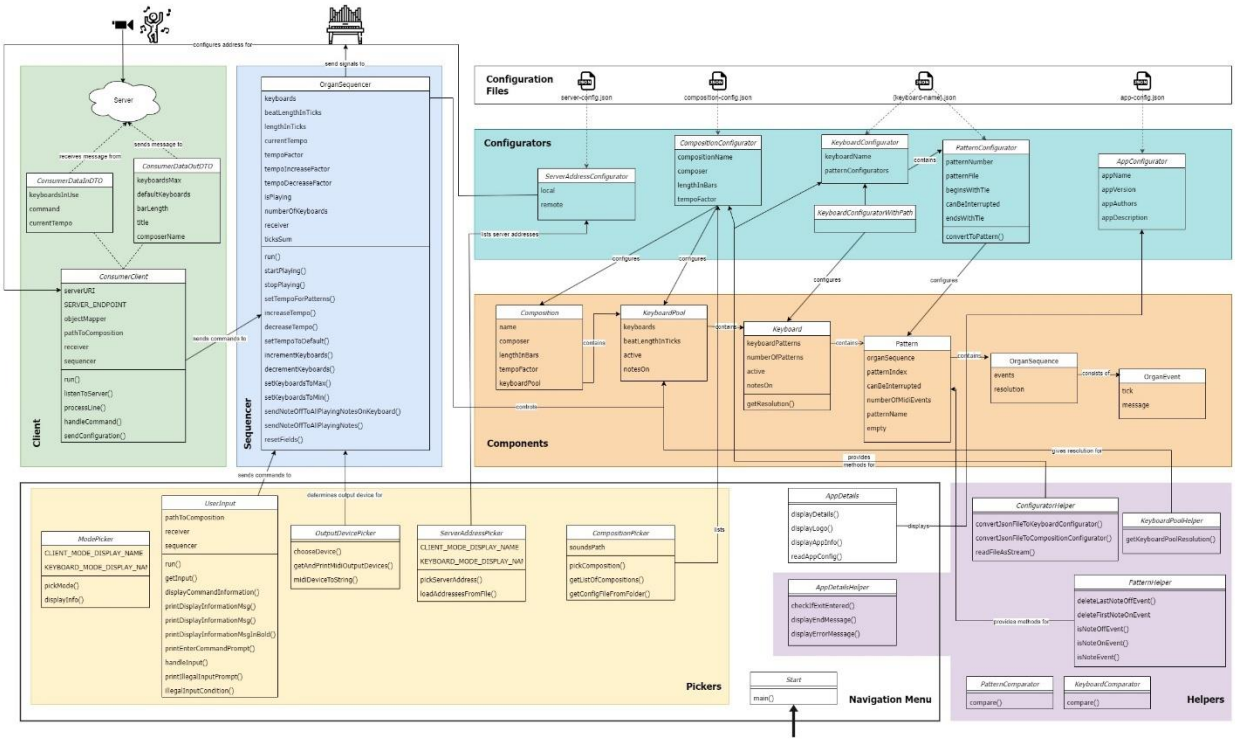
To run the application, a Java Development Kit (JDK), version 8 or later, must be installed. Additionally, Maven is required for building the application and should be installed as well.

The software does not require any third-party applications when connecting to a pipe organ. The musical instrument must only be equipped with a MIDI interface and support connection through USB or another compatible interface. However, since deploying the application with a real instrument can be challenging, it is possible to use a virtual organ for running the application. During development and testing, we used loopMIDI for opening virtual MIDI ports and GrandOrgue for simulating a real organ. As loopMIDI is only available for Windows, this posed a slight limitation in the use cases of the product. However, after consulting with Prof. Ritschel, this was considered a minor limitation, and we proceeded with the development.

Opening a virtual MIDI port with loopMIDI is straightforward. However, configuring GrandOrgue requires some familiarity with the software. There are many resources available online for configuring GrandOrgue. For specifics on MIDI channel configuration, please refer to section Installing and configuring the application.

6.4. Software architecture

The following class diagram illustrates the classes used in the implementation and the relationships between them.



The classes under the "Components" category are abstractions for real-world objects used by the sequencer. These form the foundation of the application as they are responsible for configuring the compositions, played patterns, and the keyboard/manuals of the pipe organ.

The classes under the "Configurators" category are used for deserializing configuration JSON files for application configuration, server configuration, and composition configuration.

The classes in the "Pickers" category enable the user to select an output MIDI device, a composition, the working mode of the application, and the server connection address (if in client mode) from the console. These classes effectively function as a menu for the user.

The classes under the "Helpers" category provide additional methods and functionalities for some of the base classes.

For more detailed information about each class, please refer to the Javadoc comments within the code.

6.5. Important code sections

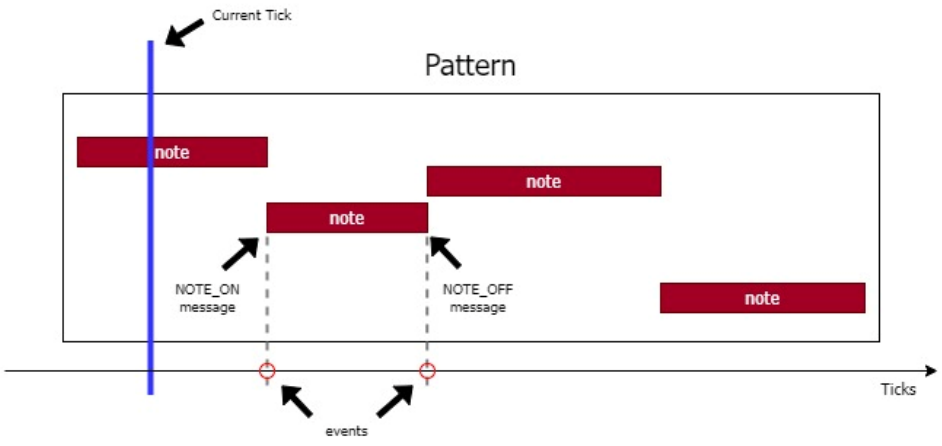
6.5.1. Sequencer

To explain how the sequencer works, let's define some important terms used in the application.

Pattern

A pattern is a sequence of MIDI messages. In our application, one pattern is created from a single MIDI file, which should be one bar in length. Each MIDI file contains a sequence of MIDI events, consisting of a MIDI message and a timestamp. The timestamp indicates when the MIDI event is executed, measured in ticks. One tick can represent different lengths of time in different files. To ensure consistent timing across all patterns in the sequencer, we use the MIDI file's metric resolution, which specifies the number of ticks per quarter note. In Java, the resolution of a MIDI file can be obtained using the getResolution() method of the Sequence class. This condition is verified by the getKeyboardPoolResolution() method in the KeyboardPoolHelper class.

A MIDI message can contain various information for controlling the output device, but we are primarily interested in the NOTEON and NOTEOFF messages. A NOTEON message is sent when a note is depressed (starts), and a NOTEOFF message is sent when a note is released (ends). Please refer to the diagram below for an example illustration of a pattern.



Keyboard

A keyboard is an abstract object that represents all the patterns to be played sequentially on a single manual of the MIDI organ. A keyboard consists of a sequence of patterns. A pipe organ can have up to 5 keyboards (4 manuals and 1 pedalboard). For naming the keyboards in our software, we used the English names for the divisions on an organ console, as follows (from bottom to top): Choir, Great, Swell, Solo/Echo,

and Pedal. The keyboard names, their respective order, and the MIDI channel on which the MIDI events for each keyboard will be sent are stored in the KeyboardName Enum.

Keyboard Pool

A keyboard pool is an abstract object that represents all the keyboards to be included in a composition.

Composition

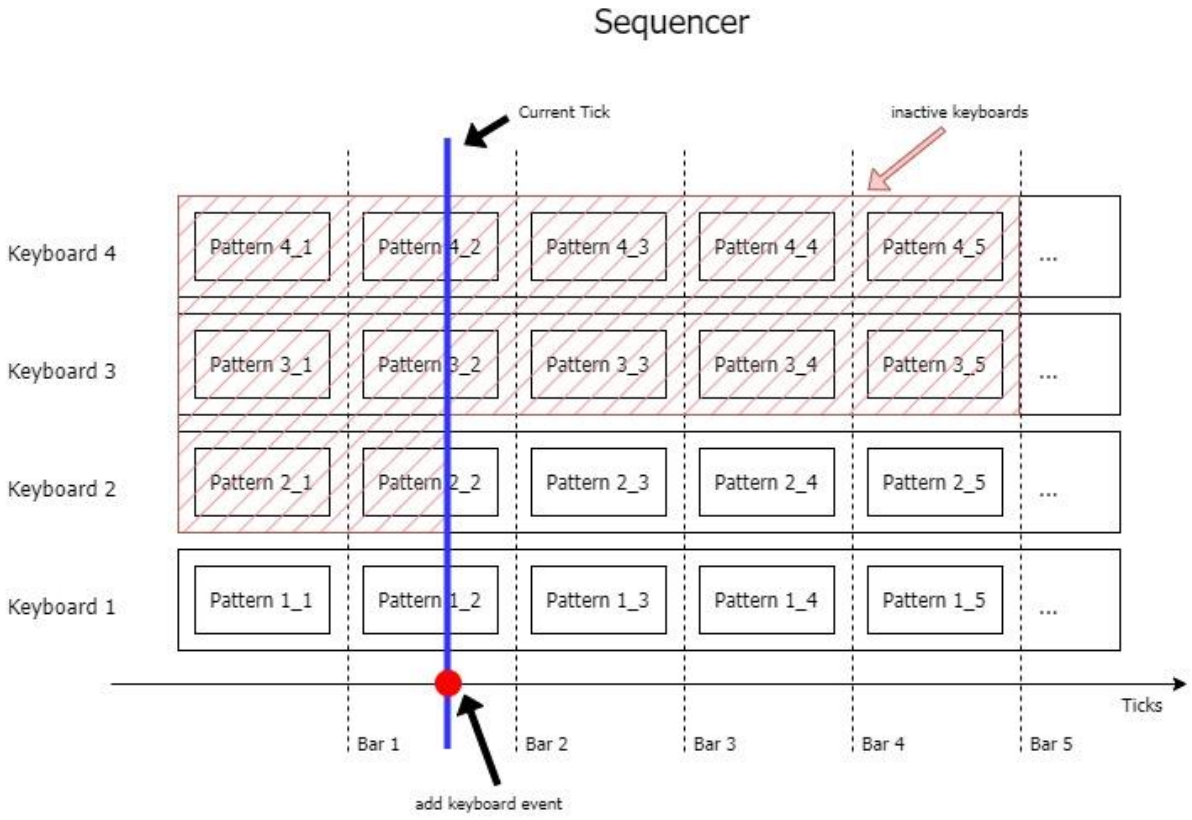
A composition is an abstract object representing a real musical piece within the context of our application. A composition consists of a keyboard pool, the length of the composition in bars, and additional information such as the title and composer. Each composition must also have a tempo change factor, which determines the extent of tempo changes in the sequencer. More about this will be explained in the following chapter.

Tempo

The tempo indicates the speed at which the music is playing. For the purposes of our project, five tempo values are available. These values are stored in the Tempo Enum and are as follows: VERY_SLOW, SLOW, NORMAL, FAST, and VERY_FAST.

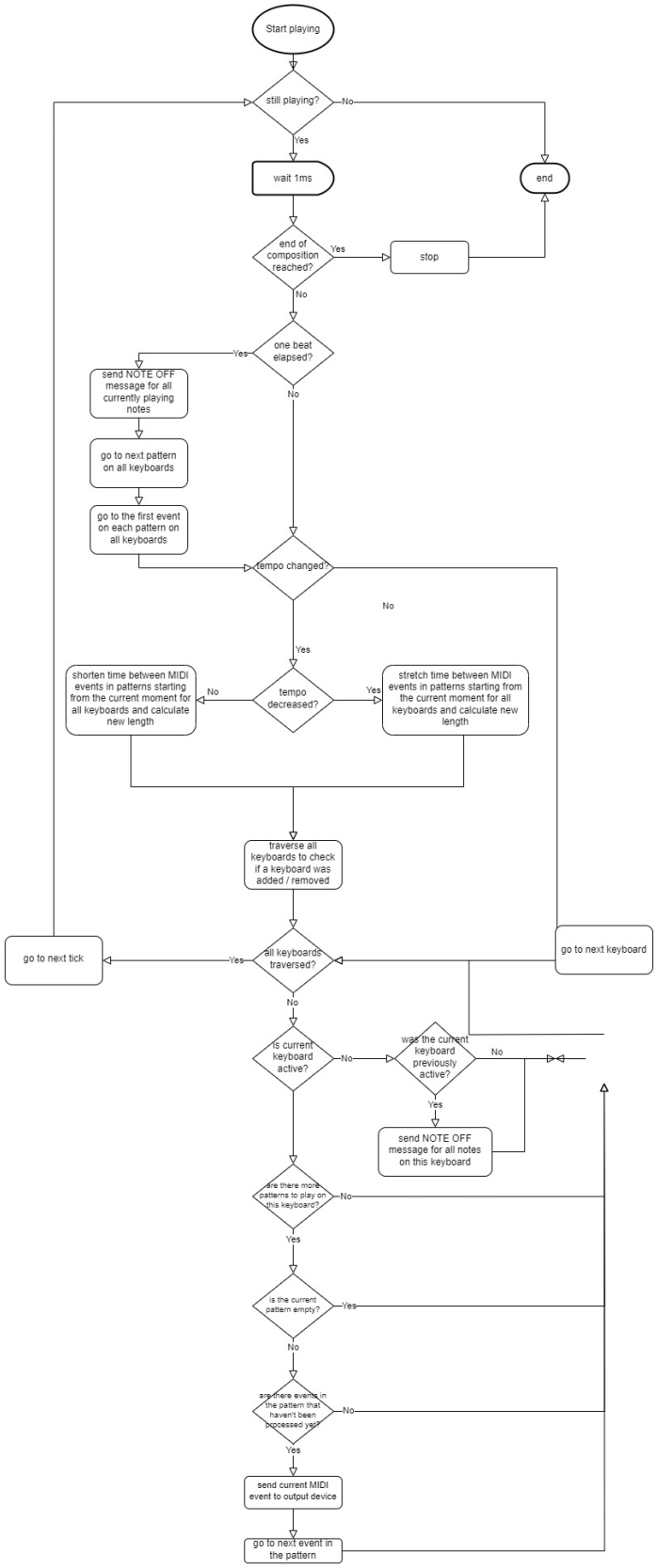
Please note that not all keyboards need to be included in a single composition. The number of keyboards in a composition is specified during its configuration. More details can be found in the section Installing and configuring the application

Sequencer



The most important part of the application is the sequencer. Its primary is to read MIDI messages from the patterns on all keyboards and send them to the output device, while dynamically adjusting the volume (dynamics) and tempo of the composition in real-time.
The fundamental function of the sequencer for managing dynamics (Crescendo/Decrescendo) involves activating or deactivating a keyboard. Technically, this means that MIDI messages in the patterns on a keyboard are either sent or not sent to the output device, making them audible or not. To simplify and unify different components of our system, we use the terms add/remote keyboards (which, for the user, translates to increasing/decreasing dynamic or crescendo/decrescendo in musical terminology).
The sequencer also allows tempo changes in real time. This change is implemented by stretching or shrinking the interval between the MIDI events in each pattern, effectively adapting the length of the bars and the overall composition to the new tempo.

The sequencer algorithm is illustrated in the following UML diagram:



This algorithm (at least its main part) is implemented in the startPlaying() method of the OrganSequencer class. Here's a walkthrough of the sequencer code:

Initialisation of variables:

```
1. long currSeqLenInTicks = this.lengthInTicks;
 2. long currBeatLenInTicks = this.beatLengthInTicks;
 3. int[] currentPatternIndex = new int[this.numberOfKeyboards];
 4. int[] currentEventIndex = new int[this.numberOfKeyboards];
 5. boolean[] previousCondition = new boolean[numberOfKeyboards];
 6. previousCondition[0] = true;
 7. Tempo previousTempoFactor = this.currentTempo;
 8. long ticks = 0;
 9. keyboards.getKeyboards().getFirst().makeActive();
```

The array currentPatternIndex represents the index of the pattern that is currently playing on each keyboard. The array currentEventIndex is used to track the current MIDI event in inside the patterns for each keyboard. The array previousCondition saves the previous condition if each keyboard (active/inactive). On line 6 the previous condition of the first keyboard is set to active, since at the beginning of the sequencer only the first keyboard is active (line 9). The variable previousTempoFactor is used for tracking the previous tempo of the sequence. At the beginning it has the value of the current tempo, which is normally set to NORMAL. The variable ticks is used for tracking the current tick inside a single bar. This value is being reset at the beginning of each bar.

The command

```
1. this.isPlaying = true;
```

Indicates that the sequencer has started playing.

```
1. while (isPlaying) {
2.         Thread.sleep(1);
3.         …
```

To enable real-time operation, the current thread is paused for 1 ms before performing additional actions.

```
1. if (ticksSum >= currSeqLenInTicks) {
2.                     System.out.println(colorize("Composition ended!", Attribute.BLUE_BACK(),
Attribute.BLACK_TEXT(), Attribute.BOLD()));
3.                     stopPlaying();
4.                 }
5.
```

These statements check if the end of the sequence is reached. As we mentioned the length of the sequence is provided in the Composition class. When an OrganSequencer

object is created this length is also calculated in MIDI ticks, so that it can be tracked by the sequencer.

Check if a bar elapsed

These statements check if one beat has elapsed. In this case all currently playing notes are released and the ticks counter, as well as the indexes of the current event for all keyboards are set to 0. The loop then checks if more patterns are left to play on each keyboard. If this is not the case, the pattern index on this keyboard is set to -1.

Detect tempo changes

```
1. if (previousTempoFactor != this.currentTempo) {
 2.                     if (previousTempoFactor.getValue() < this.currentTempo.getValue()) {
 3.                         int iterations = this.currentTempo.getValue() -
previousTempoFactor.getValue();
 4.                         for (int i = 0; i < iterations; i++) {
 5.                             setTempoForPatterns(currentPatternIndex[0], false);
 6.                         }
 7.                         currBeatLenInTicks = (long) (currBeatLenInTicks *
this.tempoDecreaseFactor);
 8.                         currSeqLenInTicks = (long) (ticksSum + (currSeqLenInTicks - ticksSum) *
this.tempoDecreaseFactor);
 9.                     } else {
10.                         int iterations = previousTempoFactor.getValue() -
this.currentTempo.getValue();
11.                         for (int i = 0; i < iterations; i++) {
12.                             setTempoForPatterns(currentPatternIndex[0], true);
13.                         }
14.                         currBeatLenInTicks = (long) (currBeatLenInTicks *
this.tempoIncreaseFactor);
15.                         currSeqLenInTicks = (long) (ticksSum + (currSeqLenInTicks - ticksSum) *
this.tempoIncreaseFactor);
16.                     }
17.                     previousTempoFactor = this.currentTempo;
18.                 }
```

The first if-statement checks if the current and previous tempo values are different. If this is the case, i.e., the tempo was changed, it determines whether the tempo was decreased (line 2) or increased (line 9). In both cases, a variable iterations is initialized. As we mentioned earlier, the Enum Tempo saves the tempo levels for the sequencer. However, it also stores the value adjustment, which is used to determine if the tempo was changed only once (e.g., from NORMAL to FAST) or twice (e.g., from VERYFAST to NORMAL). The values adjustment for each Enum value are as follows: VERYSLOW -> 2, SLOW -> 1, NORMAL -> 0, FAST -> -1, VERY_FAST -> -2. The value of iterations indicates how many times the tempo should be decreased or increased. This value is used in the following for-loop, which performs the actual tempo change.

Let us take a look at the function setTempoForPatterns():

```
1. public void setTempoForPatterns(int index, boolean increase) {
 2.     keyboards.getKeyboards().forEach(keyboard -> {
```

```
 3.         if (index != -1) {
 4.             for (int i = index; i < keyboard.getNumberOfPatterns(); i++) {
 5.                 Pattern currentPattern = keyboard.getKeyboardPatterns().get(i);
 6.                 for (int ii = 0; ii < currentPattern.getNumberOfMidiEvents(); ii++) {
 7.                     OrganEvent oldEvent = currentPattern.getOrganEvent(ii);
 8.                     MidiMessage oldMessage = oldEvent.getMessage();
 9.                     int factoredTick = (int) (increase ? oldEvent.getTick() *
this.tempoIncreaseFactor : oldEvent.getTick() * this.tempoDecreaseFactor); // updated timestamp for
event after tempo change
10.                     currentPattern.setEvent(ii, new OrganEvent(oldMessage, factoredTick));
11.                 }
12.             }
13.         }
14.     });
15. }
```

Changing the tempo involves stretching or narrowing the distance between the MIDI events in the patterns. The first argument of the method, index, specifies the starting pattern index within the keyboards from where the MIDI events should begin shifting. The second argument indicates whether the tempo was increased (true) or decreased (false). The method iterates through all patterns starting from the provided index across all keyboards. For each event in each pattern, the original event and its corresponding MIDI message are first retrieved. Next, a new timestamp for the event is calculated (line 9) by multiplying the old timestamp with a factor that reflects tempo increase or decrease. This tempo factor is determined by adding or subtracting the tempo factor from the Composition class to or from 1. Finally, the event is rescheduled using the original MIDI message and the adjusted timestamp.

Returning to the sequencer: when the tempo changes, the new length of the composition in MIDI ticks, as well as the length of a single bar, must also be updated. The latter is straightforward—simply multiply the old value by the corresponding tempo factor (lines 7 and 14). For the new length of the composition, we use the following formula:

$$length_{new} = tick_{current} + (length_{old} - tick_{current}) * factor_{tempo}$$

This formula essentially takes the current timestamp, subtracts it from the old length, multiplies the result by the corresponding tempo factor, and finally adds back the current timestamp.

Detect changes in keyboards

```
1. for (int keyboardIndex = 0; keyboardIndex < keyboards.getKeyboards().size(); keyboardIndex++) {
 2.     Keyboard currentKeyboard = keyboards.getKeyboards().get(keyboardIndex);
 3.     if (!currentKeyboard.isActive()) {
 4.         if (previousCondition[keyboardIndex]) {
 5.             sendNoteOffToAllPlayingNotesOnKeyboard(currentKeyboard);
 6.             previousCondition[keyboardIndex] = false;
 7.         }
```

```
 8.          continue;
 9.      }
10.      previousCondition[keyboardIndex] = true;
11.      if (currentPatternIndex[keyboardIndex] != -1) {
12.          Pattern currentPattern = currentKeyboard
13.                  .getKeyboardPatterns()
14.                  .get(currentPatternIndex[keyboardIndex]);
15.          if (!currentPattern.isEmpty() && currentPattern
16.                  .getOrganEvent(currentEventIndex[keyboardIndex])
17.                  .getTick() <= ticks
18.                  && currentEventIndex[keyboardIndex] < currentPattern.getNumberOfMidiEvents() -
1) {
19.              OrganEvent currentEvent =
currentPattern.getOrganEvent(currentEventIndex[keyboardIndex]);
20.              if (currentEvent.getMessage() instanceof ShortMessage sm) {
21.                  if (PatternHelper.isNoteEvent(sm)) {
22.                      if (PatternHelper.isNoteOffEvent(sm)) {
23.                          sm.setMessage(ShortMessage.NOTE_OFF,
currentKeyboard.getKeyboardName().getChannelNumber(), sm.getData1(), sm.getData2());
24.                      } else {
25.                          sm.setMessage(ShortMessage.NOTE_ON,
currentKeyboard.getKeyboardName().getChannelNumber(), sm.getData1(), sm.getData2());
26.
keyboards.getKeyboards().get(keyboardIndex).addNoteToNotesOn(sm.getData1());
27.
28.                      }
29.                      receiver.send(currentEvent.getMessage(), currentEvent.getTick());
30.
31.                  }
32.              }
33.              currentEventIndex[keyboardIndex]++;
34.          }
35.      }
36. }
```

This part of the sequencer traverses all keyboards to check if a new keyboard was added or removed. The first two if-statements check if the current keyboard, previously active, is now set to inactive, indicating that the keyboard must be removed. If this is the case, the previous condition of the keyboard is set to inactive and the loop continues to the next keyboard.

If the current keyboard is active, the previous condition for the keyboard is updated to active. In this case, the current pattern for the keyboard is retrieved (lines 12-14). We then verify if the pattern is not empty and if the current event in the pattern is within the current pattern and bar. If both conditions are met, we retrieve the event (line 19). We then look for NOTE_ON and NOTE_OFF messages in the event. If one is found, it is first converted to ShortMessage. This conversion is necessary to send the message to a specific channel. The channel for each keyboard is taken from the Enum KeyboardName. The message is then sent to the output device with the current timestamp (line 29). Finally, we proceed to the next event by incrementing the event index value by one.

```
1. ticks++;
2. ticksSum++;
```

Lastly, we advance to the next tick within the bar as well as the next tick in relation to the total length of the composition.

6.6. Installing and configuring the application

To run the application, you need a Java Development Kit (JDK) and Maven. There are two ways to start the application: through an Integrated Development Environment (IDE) or by building a JAR file.

Running the Application Through an IDE

You can start the application directly from an IDE like IntelliJ IDEA by running the Start class.

Building and Running a JAR File

Alternatively, you can build a JAR file containing all necessary dependencies and run the application from the command line. The following steps assume you are in the root directory of the client application, which is the MIDI/OrganPlayer folder.

1. Build the project
   `mvn clean generate-sources package`
2. Copy the generated jar to the root directory of the client application
   `cp .\target\dancing-pipes-jar-with-dependencies.jar`
3. Run the application
   `java -jar dancing-pipes-jar-with-dependencies.jar`
4. Clean the Build (Optional)
   You can clean the build directory, which was created in Step 1, by running the following command:
   `mvn clean`

Project File Structure and Configuration

The root folder contains two folders and two JSON configuration files, apart from the folder src holding the source code. The file `app-config.json` contains metadata about the application, such as the name, version, authors, and a brief description. This information is displayed in the terminal when starting the application. The folder assets includes a text file containing an ASCII logo for the application, which is also displayed when the application starts.

The file `server-config.json` holds the local and remote addresses for connecting to the server. Users can choose the address depending on where the server is hosted (localhost or university network).
A key part of the project file structure is the folder sounds. This is the place where the compositions and the respective MIDI files for the patterns are held and managed. Each subfolder within sounds contains a composition, only recognized as such by the presence of a `composition-config.json` file. The folder name is not important and can be freely chosen. The file `composition-config.json` must include the following information:

- Title of the composition
- Composer
- Length of the composition in bars (as integer, not string)
- Tempo factor (as double, not string). This value is used to determine the scale in which the tempo can change on one step (when increasing or decreasing).

Each composition folder must also contain sub-folders for the keyboards used in that composition. These subfolders contain the actual MIDI files. The names of the sub-folders can be chosen freely (in the example composition we have used first, second... for easier recognition, but other approaches are also possible).
Each subfolder must contain a JSON file used to configure the keyboard. It is important that there is only one JSON file in each keyboard folder, the name of the JSON file can be chosen freely by the user. This file contains the following information

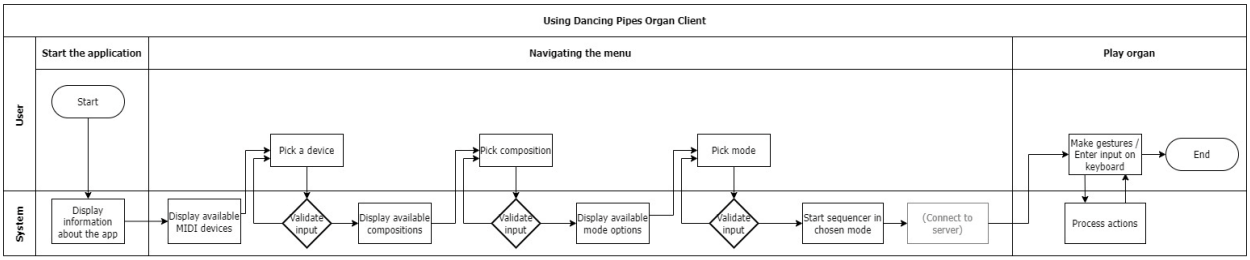
- Keyboard name. Only values from the KeyboardName Enum are allowed in this field, i.e. pedal, choir, great, swell, solo.
- Array of Pattern configuration objects. Each Pattern configuration object must contain the following information
    - Pattern number - the index of the pattern in the sequence of patterns for this keyboard as an integer (e.g. first, second...)
    - Pattern file - the filename of the MIDI file for this pattern. The MIDI file must be in the same folder as the JSON file. It is also possible to leave this field empty. In this case the pattern will be skipped by the sequencer.
    - Boolean values indicating whether the file begins or ends with a tie. In music notation, a tie is a curved line connecting the heads of two notes of the same pitch, indicating that they are to be played as a single note with a duration equal to the sum of the values of the individual notes. If a pattern ends with a tie, the next pattern should begin with a tie. In this case, the sequencer won't play the two tied notes twice, but only once.
    - Boolean value indicating whether the pattern can be interrupted immediately (true) or whether it should be interrupted at the start of the

next bar. This is not implemented in the current version of the application, so this value is ignored by the sequencer.

Only files listed in the JSON file will be recognized as patterns and played by the sequencer. Each pattern must have a length of one bar. If more files are listed than the length of the composition, the remaining patterns will be ignored. If a keyboard contains fewer patterns than the length of the composition, ensure to list empty patterns to match the composition's length. Failure to do so will result in unwanted behaviour from the sequencer.

6.7. Operating the application

The following diagram shows the normal operation of the application:



The user can navigate through the application using the menu on the terminal. If the user enters "exit", the application is terminated. This does not apply if the client is connected to the server. In this case, the user must exit the application manually by pressing `Ctrl+C`.

The application can be launched in two modes: Keyboard input mode and client mode. In the former, the user can control the sequencer from the keyboard. This mode is useful for testing purposes, as it doesn't require the other components of the system to be started. Accordingly, changes made to the sequencer in this mode won't be reflected on the front-end. The client mode connects to the server at the address selected by the user. It then waits for a start command from the server and starts the sequencer. The client implementation and the DTOs used to communicate with the server can be found in the server folder in `src`.

6.8. Current limitations and future improvements

The current version of the application meets the basic requirements for our project. However, there is potential for future enhancements to make the application more flexible. Areas for improvement include:

- Allowing patterns to have lengths other than one bar.
- Allowing the user to manually choose the tempo by entering a value in beats-per-minute (BPM).

- Enabling or disabling the keyboard, or changing the tempo, only when the bar changes.
- Improving the keyboard configuration process.

7. Hand gesture recognition *(By Matsvei)*

Attention! This model is not used in the current version of the project, but later it can be combined with a body recognition model. This chapter was created for future changes.

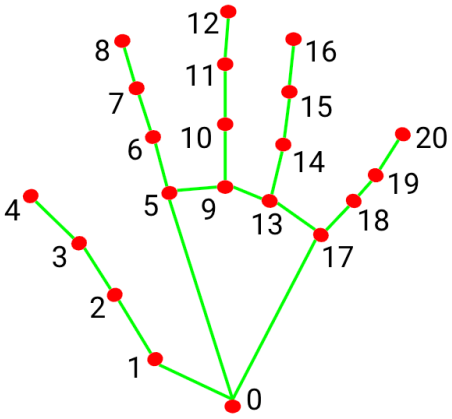
7.1. General overview

As an alternative, it was decided to implement hand gesture recognition. It provides tools for hand sign and finger gesture recognition using MediaPipe. The models for recognition are implemented using TensorFlow, and data can be trained and collected using provided scripts and notebooks.

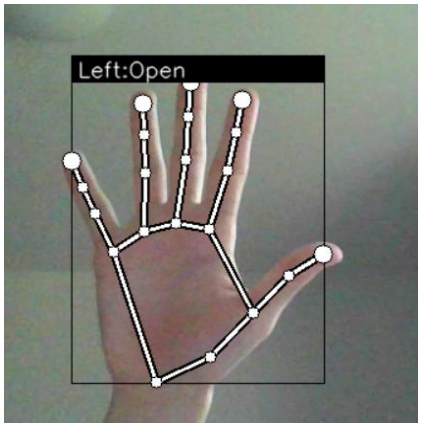
Hand recognition is divided into two parts: **gesture recognition** and **index finger tracking**. Both parts use the sequential neural network model from tensorflow.

Gesture recognition

keypoint_classification.ipynb trains the model based on input data (42 numbers - coordinates of 21 key points (X and Y of each key point)). Key points are indicated below in the picture.



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP



Each point is displayed on the screen and the territory of the hand is also shown with frames. The name of the gesture is written at the top. It also displays which hand (left or right) is on the camera.
There are currently three gestures available(close,open,pointer) . It is also possible to add your own gestures

How to add your own gesture?

1) Please open the file *keypoint_classifier_label*
2) Write the name of your gesture
3) Remember the serial number of your gesture (remember that the countdown starts from zero)

4) Start app.py
5) press k (you will go to the mod for installing new gestures).
6) Select a hand pose for the gesture and press a number equal to the serial number of the gesture that you added a couple of steps ago
   **please do as many different poses as possible for one gesture, since when you press a number, it is the coordinate data at the moment you press the number that is remembered*
7) Restart app.py

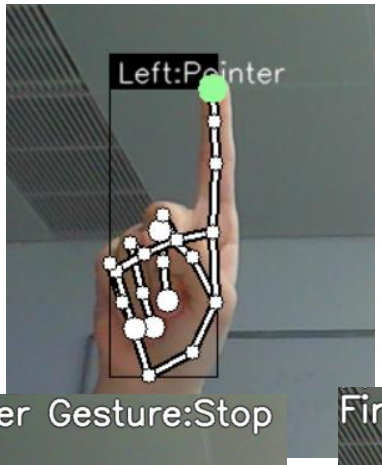
How it works?

In file *keypoint_classifier_label* are indicated possible labels for gestures. In file *keypoint* there is a table where the first number is the serial number of the label as well as the coordinates of the pose belonging to this label. When you press a number in the mode of adding gestures, the camera captures this pose at the moment and the table is supplemented with the pressed number + coordinates of this pose

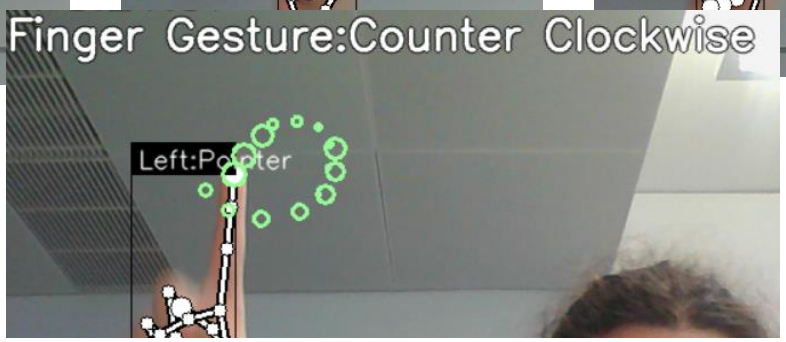
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1519	0	0	0	-0.21659	0.073733	-0.34101	0.253456	-0.40553	0.419355	-0.40092	0.552995	-0.28571	0.198157	-0.35945	0.479263	-0.37327	0.645161	-0.36866
1520	0	0	0	-0.2287	0.080717	-0.33632	0.255605	-0.38565	0.426009	-0.36323	0.565022	-0.26906	0.179372	-0.33184	0.452915	-0.34978	0.627803	-0.35426
1521	0	0	0	-0.16889	0.048889	-0.21778	0.217778	-0.24444	0.4	-0.24889	0.551111	-0.08	0.151111	-0.06222	0.377778	-0.02667	0.524444	0.013333
1522	0	0	0	-0.16114	0.066351	-0.22275	0.236967	-0.25118	0.417062	-0.24171	0.559242	-0.19431	0.194313	-0.2654	0.469194	-0.2891	0.649289	-0.3128
1523	1	0	0	-0.3	-0.18667	-0.44667	-0.48	-0.46667	-0.76	-0.46667	-1	-0.3	-0.67333	-0.29333	-0.90667	-0.31333	-0.66667	-0.31333
1524	1	0	0	-0.32432	-0.17568	-0.5	-0.4527	-0.5473	-0.74324	-0.56757	-1	-0.41216	-0.65541	-0.39865	-0.91892	-0.38514	-0.67568	-0.38514
1525	1	0	0	-0.33803	-0.16901	-0.54225	-0.43662	-0.59859	-0.73239	-0.61972	-1	-0.4507	-0.67606	-0.43662	-0.93662	-0.41549	-0.6831	-0.41549
1526	1	0	0	-0.34286	-0.15	-0.55	-0.42857	-0.62857	-0.72857	-0.66429	-1	-0.49286	-0.66429	-0.47857	-0.93571	-0.44286	-0.67857	-0.43571

Index finger tracking

When the hand is in the "index finger up" pose, the pointer pose is activated, which follows the movement of the index finger, displaying a trace



There are four states available in this mode. "Stop" state when the index finger is not moving, "Move" state when the index finger is moving, "Clockwise" state when the finger is moving clockwise, "Counter clockwise" state when the finger is moving counterclockwise



Directory

```
|  app.py
|  keypoint_classification.ipynb
|  point_history_classification.ipynb
|
├─model
|  ├─keypoint_classifier
|  |  |  keypoint.csv
|  |  |  keypoint_classifier.hdf5
|  |  |  keypoint_classifier.py
|  |  |  keypoint_classifier.tflite
|  |  └─ keypoint_classifier_label.csv
|  |
|  └─point_history_classifier
|      |  point_history.csv
|      |  point_history_classifier.hdf5
|      |  point_history_classifier.py
|      |  point_history_classifier.tflite
|      └─ point_history_classifier_label.csv
|
└─utils
    └─cvfpscalc.py
```

app.py. This is a sample program for inference. In addition, learning data (key points) for hand sign recognition, You can also collect training data (index finger coordinate history) for finger gesture recognition.

keypoint_classification.ipynb. This is a model training script for hand sign recognition.

point_history_classification.ipynb. This is a model training script for finger gesture recognition.

model/keypoint_classifier. This directory stores files related to hand sign recognition.The following files are stored.

- Training data(keypoint.csv)
- Trained model(keypoint_classifier.tflite)
- Label data(keypoint_classifier_label.csv)
- Inference module(keypoint_classifier.py)

model/point_history_classifier. This directory stores files related to finger gesture recognition.The following files are stored.

- Training data(point_history.csv)
- Trained model(point_history_classifier.tflite)

- Label data(point_history_classifier_label.csv)
- Inference module(point_history_classifier.py)

utils/cvfpscalc.py. This is a module for FPS measurement.

How to run?

1) please install all requirements:
    - mediapipe 0.8.1
    - OpenCV 3.4.2 or Later
    - Tensorflow 2.3.0 or Later
2) Run “python app.py”

7.2. An in-depth look at gesture recognition

- ***keypoint_classification***

This script demonstrates the complete process of preparing, training, evaluating, and converting a neural network using the TensorFlow library. Below is a step-by-step explanation of the key parts of the code:

1. Import Libraries
First, the necessary libraries are imported: csv, numpy, tensorflow, sklearn.model_selection, pandas, seaborn, matplotlib, and sklearn.metrics.

2. Initialization and Data Loading
A random seed is set for reproducibility (RANDOM_SEED). The data is loaded from a CSV file (keypoint.csv). The data represents keypoint coordinates, with columns 1 to 42 used as features (X_dataset) and the first column as class labels (y_dataset).

3. Splitting Data into Training and Testing Sets
Using the train_test_split function, the data is split into training and testing sets in a 75% to 25% ratio, respectively.

4. Model Creation
A Keras Sequential model is created with three fully connected layers:

The first layer: an input layer with 42 neurons (corresponding to the number of features), followed by a Dropout layer for regularization.
The second layer: 20 neurons with ReLU activation and a Dropout layer.
The third layer: 10 neurons with ReLU activation.
The output layer: 4 neurons (corresponding to the number of classes) with softmax activation.

5. Model Compilation
The model is compiled with the Adam optimizer and sparse_categorical_crossentropy loss function. The evaluation metric is accuracy.

6. Model Training
The model is trained for 1000 epochs with a batch size of 128. The validation data is the test set. During training, cp_callback and es_callback are used for model checkpointing and early stopping.

7. Model Evaluation
The model is evaluated on the test data, after which the saved model is loaded.

8. Inference (Prediction)
For inference testing, the model predicts the class for one example from the test set. The class probability distribution and the predicted class are output.

9. Visualization and Analysis
A confusion matrix is created and displayed to visualize the model's performance. Additionally, a classification report with key metrics is printed.

10. Saving and Converting the Model
The model is saved without the optimizer for inference purposes. The model is then converted to TensorFlow Lite format with quantization for optimization.

11. Inference with TensorFlow Lite
The TFLite model is loaded, and tensors for input and output data are allocated. Inference is performed, and the results are output.

Summary
This script represents a comprehensive machine learning process from data loading and preprocessing to model training, evaluation, and conversion for optimized use on mobile and embedded devices with TensorFlow Lite.

- ***keypoint_classifier***

This second script defines a Python class KeyPointClassifier for performing inference using a TensorFlow Lite model. Below is a step-by-step explanation of the key parts of this script:

1. Import Libraries
The necessary libraries numpy and tensorflow are imported.
2. Class Definition: KeyPointClassifier
A class KeyPointClassifier is defined to encapsulate the functionality for keypoint classification using a TensorFlow Lite model.
3. Constructor: __init__

The __init__ method initializes the classifier with a given TensorFlow Lite model path and the number of threads for inference.
The TensorFlow Lite model is loaded using tf.lite.Interpreter.
Tensors for the model are allocated using self.interpreter.allocate_tensors().
Input and output details of the model are stored in self.input_details and self.output_details respectively.

4. Inference Method: __call__

The __call__ method performs inference on the provided list of landmarks.
The input tensor is set with the provided landmark data.
Inference is executed using self.interpreter.invoke().
The output tensor is retrieved and the index of the maximum score is determined using np.argmax.

Interaction Between the Two Scripts

The two scripts are connected in a machine learning workflow where the first script handles model training, evaluation, and conversion, while the second script utilizes the trained and converted TensorFlow Lite model for inference. Here's how they interact:

Model Training and Conversion (*keypoint_classification*)

The first script trains a neural network model on keypoint data, evaluates it, and then saves the model in TensorFlow Lite format (keypoint_classifier.tflite).

Inference with TensorFlow Lite (*keypoint_classifier*)

The second script defines a KeyPointClassifier class that loads the saved TensorFlow Lite model and performs inference on new keypoint data.

6.3 An in-depth look at index finger tracking

- ***point_history_classification***

This script performs the complete process of preparing, training, evaluating, and converting a neural network using TensorFlow and TensorFlow Lite for a point history classification task. Here's a detailed explanation:

1. Import Libraries

The script imports essential libraries for data manipulation (csv, numpy, pandas), machine learning (tensorflow, sklearn), and visualization (seaborn, matplotlib).

2. Initialization and Data Loading

1. **Dataset and Model Paths**: Paths for the dataset and the model save location are defined.
2. **Constants**: Number of classes (NUM_CLASSES), number of time steps (TIME_STEPS), and dimensionality of input data (DIMENSION) are set.
3. **Loading Data**: The dataset is loaded from a CSV file where columns 1 to 32 (for 16 time steps and 2 dimensions) are used as features, and the first column is used as the class label.

4. **Splitting Data**: The data is split into training and testing sets using a 75% to 25% ratio.

3. Define the Model Architecture

1. **Model Selection Flag**: A flag (use_lstm) is set to determine whether to use an LSTM-based model or a Dense model.
2. **LSTM Model**: If the flag is set to true, an LSTM-based model is defined with input reshaping, LSTM layers, Dropout layers, and Dense layers.
3. **Dense Model**: If the flag is false, a simpler Dense model is defined with Dropout layers and Dense layers.

4. Compile and Train the Model

1. **Callbacks**: Callbacks for model checkpointing (to save the best model) and early stopping (to prevent overfitting) are defined.
2. **Compilation**: The model is compiled with the Adam optimizer and sparse_categorical_crossentropy loss function.
3. **Training**: The model is trained for up to 1000 epochs with a batch size of 128. Validation data is used to monitor performance during training.

5. Evaluate the Model and Make Predictions

1. **Loading Best Model**: After training, the best model is loaded from the saved file.
2. **Predictions**: Predictions are made on the test data. The predicted class probabilities and the most likely class are printed.

6. Print Confusion Matrix and Classification Report

1. **Confusion Matrix Function**: A function is defined to print the confusion matrix and the classification report, which provides detailed performance metrics.
2. **Predictions on Test Data**: The model makes predictions on the entire test dataset, and the confusion matrix and classification report are generated and displayed.

7. Convert to TensorFlow Lite

1. **Saving the Model**: The trained model is saved without the optimizer for inference.
2. **Conversion**: The model is converted to TensorFlow Lite format with quantization to optimize for size and performance.
3. **Saving TFLite Model**: The converted TensorFlow Lite model is saved to a file.

8. Inference with TensorFlow Lite

1. **Load TFLite Model**: The TensorFlow Lite model is loaded and tensors are allocated.
2. **Inference**: The model performs inference on a single test sample. The predicted class probabilities and the most likely class from the TensorFlow Lite model are printed.

- ***point_history_classifier***

This script defines a PointHistoryClassifier class for performing inference using a TensorFlow Lite model that has been trained to classify sequences of points (point history). Below is a detailed explanation of the key components and how it interacts with the first script:

1. Import Libraries

The script imports the necessary libraries: numpy for numerical operations and tensorflow for loading and running the TensorFlow Lite model.

2. Class Definition: PointHistoryClassifier

The PointHistoryClassifier class is defined to encapsulate the functionality for point history classification using a TensorFlow Lite model.

3. Constructor: __init__

The __init__ method initializes the classifier with the given parameters:

- **model_path**: Path to the TensorFlow Lite model file.
- **score_th**: Threshold for classification scores to consider a result valid.
- **invalid_value**: Value to return if the classification score is below the threshold.
- **num_threads**: Number of threads to use for inference.

The method performs the following steps:

- **Load TensorFlow Lite Model**: Loads the model using tf.lite.Interpreter.
- **Allocate Tensors**: Allocates tensors for the model.
- **Retrieve Input/Output Details**: Stores the input and output tensor details for later use.

4. Inference Method: __call__

The __call__ method performs inference on the provided point history data:

- **Set Input Tensor**: Sets the input tensor with the provided point history data.
- **Invoke Inference**: Calls the invoke method to run inference.
- **Retrieve Output Tensor**: Retrieves the output tensor from the model.
- **Determine Result Index**: Finds the index of the maximum score in the output tensor.
- **Apply Score Threshold**: Checks if the maximum score is below the threshold. If it is, returns the invalid_value; otherwise, returns the index of the classification result.